Simulation and Comparative Analysis of Machine-Learning-Assisted Surface-Code Decoders

A Report

Submitted by

PRETHISH KUMAR M B

Ra2311023010002

Department of Physics and Nanotechnology

SRM Institute of Science and Technology, Chennai.

Department of Physics and Nanotechnology

SRM Institute of Science and Technology, Chennai.

Under the supervision of

Dr. KARTHIKA P C

Asst.Professor

Department of Physics and Nanotechnology

SRM Institute of Science and Technology, Chennai.



SRM Institute of Science and Technology, Kattankulathur, Chennai

July 2026.

BONAFIDE CERTIFICATE

Certified that this project report titled "**Simulation and Comparative Analysis of Machine-Learning-Assisted Surface-Code Decoders"** is the Bonafide work of **Prethish Kumar M B** [**Reg No: RA2311023010002**], who has carried out his project work under my supervision. Certified further, that to the best of my knowledge, the work reported herein does not form any other project report or dissertation.

(Signature of H.O.D)

Dr. E. Senthil Kumar
Head of the Department
Department of Physics
SRMIST.
Chennai

(Signature of Guide)

Dr. Karthika P C
Asst.Professor
Department of Physics
SRMIST
Chennai

Signature of the Internal Examiner(s)

ABSTRACT

The framework needed to move from noisy physical quantum devices towards fault-tolerant, reliable quantum computation is quantum error correction (QEC). Of many strategies, a particularly promising one for achieving fault tolerance is the surface code, which performs repeated stabilizer measurements locally that detect physical noise patterns without interfering with the encoded data. The decoding algorithm that is responsible for converting raw measurement results into precise logical corrections, however, is the key to the ultimate error suppression ability of these architectures. In this work, we explore the possibility of data-driven error correction by systematically comparing the performance of the conventional graph-theoretic decoder and the machine-learning-based surface-code decoder through computation.

We use our own experimental setup to simulate a memory circuit based on surface code with a code distance (d=3) using three rounds of syndrome measurements, with a fixed physical error probability of 0.5%. We used this environment to generate a dataset of 50,000 independent simulated shots and split it into clearly defined training, validation and testing pipelines. We tested and evaluated four different decoding frameworks: Minimum Weight Perfect Matching (MWPM) using PyMatching, Logistic Regression (LR), class-balanced Logistic Regression (LRCB), and a feed-forward Multi-Layer Perceptron (MLP) using scikit-learn. All four decoders were tested on the same, completely separated set of 7,500 samples, and the absolute logical failure rate was used as the measure of the engineering success.

The empirical results show the following ranking under the tested configuration. (1) MWPM achieved the lowest logical failure rate, with 144 failures among 7,500 test shots (1.92%). (2) The nonlinear MLP achieved a logical failure rate of 2.48%. (3) Standard Logistic Regression and class-balanced Logistic Regression achieved failure rates of 10.76% and 20.77%, respectively. The MLP performed substantially better than the two linear classifiers but did not outperform the MWPM baseline. These findings apply to the tested distance-3 surface-code configuration and should not be generalized beyond the evaluated conditions.

Finally, this thesis presents an end-to-end computational workflow for benchmarking data-driven decoders for quantum error correction. The results provide a baseline for further investigations of decoding performance across code distances, physical error rates, random seeds, and runtime conditions.

Index

ABSTRACT *3*
CHAPTER 1 — INTRODUCTION *7*
1.1 Background and Motivation **7**
1.2 Research Motivation **7**
1.3 The Research Problem and Objectives **8**
1.4 A Summary of the Proposed Model **8**
1.5 Scope of the Study **9**
CHAPTER 2: THEORETICAL BACKGROUND *10*
2.1 Quantum Errors and the Need for Error Correction **10**
2.2 Surface Codes **10**
2.3 Syndrome Extraction and Logical Errors **11**
2.4 Minimum-Weight Perfect Matching (MWPM) **12**
2.5 Machine Learning for Quantum Error Decoding **13**
2.6 Comparative Evaluation Framework **14**
2.7 Scope and Limitations of the Approach **14**
2.8 Chapter Summary **15**
CHAPTER 3: LITERATURE REVIEW *16*
3.1 Development of Surface-Code Decoding **16**
3.2 Minimum-Weight Perfect Matching **16**
3.3 Machine-Learning-Based Decoding **17**
3.4 Comparison Conventional and Machine-Learning Approaches. **17**
3.5 Research Gap and Motivation of Current Study **18**
3.6 Chapter Summary **18**
Chapter 4: COMPUTATIONAL TOOLS & METHODOLOGY *19*
4.1 Research Methodology **19**
4.2 Computational Tools **19**
4.3 Surface-Code Circuit Configuration **19**
4.4 Dataset Generation and Preparation **20**
4.5 Implementation of the Decoders **21**
4.6 Performance Evaluation Metrics **23**
4.7 Experimental Consistency and Reproducibility **24**
4.8 Chapter Summary **24**
CHAPTER 5: RESULTS AND ANALYSIS *25*
5.1 Introduction **25**
5.2 Experimental Configuration **25**

5.3 Benchmarking Decoder Performance **26**
5.4 Analysis Of MWPM Decoder **27**
5.5 Logistic Regression Models **28**
5.6 Analysis of MLP Decoder **28**
5.7 Comparison Between MWPM and MLP directly. **29**
5.8 Reproducibility of Independent Random Seeds. **29**
5.9 Benchmarking Of Various Code Distances: **30**
5.10 Testing Computational Performance & Decoding Speed of algorithms **30**
5.11 Overall Discussion **31**
5.12 Limitations of the Current Results **31**
5.13 Chapter Summary **32**
CHAPTER 6: DISCUSSION ***33***
6.1 Overview of the findings. **33**
6.2 Benchmarking of Decoders **33**
6.3 Interpretation results from the Machine Learning Models **34**
6.4 Research Significance and Contribution **35**
6.5 Methodological Strengths **35**
6.6 Limitations of the Present Study. **36**
6.7 Future Research Directions **37**
6.8 Chapter Summary **37**
Chapter 7 Conclusion & Future Scope ***39***
7.1 Conclusion **39**
7.2 Achievement of Research Objectives. **40**
7.3 Research Contribution **40**
7.4 Future Scope **41**
7.5 Final Remarks **42**
REFERENCES ***43***
Appendix ***45***

List Of Figures

i. **Figure 1:-** The figure illustrates the workflow of the Quantum Circuit Simulation and supervision using Data Science (page No :- 7)

ii. **Figure 2:-** Surface code Lattice with distance (d=3,5,7 & 9) which illustrates the relationship between the code distance and the no.of correctable errors(page No :- 9)

iii. **Figure 3:-** Simple circuit illustrating stabilizer measurement and syndrome extraction in the surface code(page No :- 10)

iv. **Figure 4:-** It illustrates how an error changes the measurement syndrome(page No :- 11)

v. **Figure 5:-** Heatmap Visualization of the Detector Event Samples Generated from Surface-Code Simulation (page No :- 19)

vi. **Figure 6: Comparison of test-set logical failure rates for MWPM, standard Logistic Regression, class-balanced Logistic Regression and MLP at (d=3) and (p=0.5%).** (page No :- 25)

vii. **Figure 7**: Logical failure rate of MWPM decoder as a function of physical error probability. (page No :- 26)

viii. **Figure 8:** Logical failure rate distribution for MWPM and MLP across five independent random seeds(page No :- 28)

ix. **Figure 9:-** Comparison of Logical Failure rates for MWPM and MLP inference(page No :- 29)

x. **Figure 10:** Comparison of throughput and decoding latency for MWPM and MLP inference. (page No :- 30)

CHAPTER 1 — INTRODUCTION

1.1 Background and Motivation

In contrast to the classic binary nature of bits, quantum computing is progressing to a paradigm shift in information processing. Quantum computers are based on the principles of quantum mechanics, such as superposition, quantum interference and entanglement, and present fundamentally new possibilities to solve computational problems that are currently impossible for classical computers.

However, this is a huge promise that is severely limited by physical implementation – quantum fragility. The effects of environmental noise, imperfect control operations, and error in measurement, on qubits are significant. Without being dealt with, these flaws rapidly ruin delicate quantum states and upset calculations. The critical missing ingredient to counteract such a weakness is quantum error correction (QEC). Rather than storing logical information on a single qubit, QEC spreads it out among a group of physical qubits. It uses carefully designed, non-destructive measurements to sense signatures of noise, or error syndromes, without actually observing and destroying the logical state of the encoded data.

The challenge is naturally related with the broader challenge of controlling physical systems at the atomic and microscopic scale from a nanotechnology standpoint. In these sizes, the behavior of the system is greatly influenced by the environmental interactions. This study does not use materials synthesis or fabrication of devices but takes this physical-sciences perspective on the reliability of quantum information, regarding the decoding process as an essential control layer in sub-micron physical architectures.

To date, the surface code is one of the more promising candidates for fault-tolerant architectures, in part because it uses local stabilizer measurements that are placed in a two-dimensional array. These point measurements provide a continuous time series of syndrome data. This raw data is, however, only a diagnostic report and must now be understood using a decoding algorithm that will be used to analyse the syndromes, identify the actual physical faults, and correct them. In the end, the reliability of the fault-tolerant quantum computer will depend on how fast and precise the decoder is.

1.2 Research Motivation

Surface codes have been conventionally decoded for years using a technique called Minimum-Weight Perfect Matching (MWPM). MWPM translates error syndromes into a graph, and looks for the lowest-weight, most likely combination of physical errors which could have caused the observed fault. It is well known and highly optimized and thus is used as a benchmark for any new decoding method. Recently, machine learning has emerged as an intriguing alternative. Rather than using a fixed geometric algorithm, a machine learning model can be trained using simulated data to identify complicated, underlying patterns between the syndrome patterns and logical outcomes. These models can then make logical inferences about new data points almost instantaneously after training.

A more complex model does not, of course, necessarily lead to better decoding, however. The effectiveness of the machine learning model in real-world applications is crucial, depending on the quality of the training data, the structure of the model, its ability to manage class imbalance, and the type of noise patterns it faces. It cannot simply be assumed that machine learning will

outperform traditional algorithms. Its real value can only be known when compared side-by side with MWPM, using the same conditions and measures.

This project examines just that comparison. We seek to systematically assess the current state of the art of machine learning compared to MWPM by combining supervised machine learning models with a simulated surface-code circuit with no presupposed superiority of machine learning.

1.3 The Research Problem and Objectives

The main question this research aims to answer is whether there is a trade-off between specific supervised machine learning models and MWPM that can be made in terms of logical decoding performance when taken against a realistic simulated noise model.

To do this, we consider three very different types of machine learning, which vary in complexity: ordinary logistic regression, class-balanced logistic regression, multi-layer perceptron (MLP). Logistic regression is a clear, linear base line. To handle some of these potential skews in the frequency of error labels when training, the class-balanced logistic regression is used to adjust this baseline. Finally, the MLP adds the element of interconnection among sets of artificial neurons, enabling the system to learn the complex and nonlinear relationships that lurk between the inputs of the detectors and the logical-observable outputs.
The project has six specific objectives, identified to guide this investigation:

1. Simulate controlled surface-codes to generate circuit samples and error syndrome data in the framework Stim.
2. Use a traditional decoding baseline through PyMatching to run MWPM.
3. Train and tune the three machine learning classifiers used (Ordinary Logistic Regression, Class Balanced Logistic Regression and MLP).
4. Logically compare the decoding performance of all four models in terms of logical failure rate and classification metrics.
5. Test the stability of the performance of the decoders using stress tests with different code distances, different physical error probabilities and independent random seeds.
6. Under standard computational conditions, measure and record operational metrics, such as decoding latency and throughput.

1.4 A Summary of the Proposed Model

This study is designed on a framework that connects quantum circuit simulation and supervised data science. The workflow is divided into 5 different steps:



Figure 1:- *The figure illustrates the workflow of the Quantum Circuit Simulation and supervision using Data Science*

The simulator in this circuit simulation application is used to simulate various datasets, which are subsequently subjected to the model training and validation process. The resulting model is

compared with different decoders to analyze performance. We build a surface code circuit and simulate its execution with Stim for a chosen noise model. Our simulated runs produce a sequence of detector outcomes (our features) and a logical-observable outcome (our label).

Then, this generated data is split up into training, validation, and test sets. The training data is used to train and map the parameters of the machine learning models, and the validation subset is used to fine-tune the hyperparameters (e.g., by tuning the decision threshold of the MLP). Most important of all, test data is completely separate and available only for a final evaluation.
After training, the models compete with MWPM with the same, unseen test samples. Since the ultimate goal of error correction is to protect the logical data, we use the error failure rate as our main measure of success. To fully comprehend the success or failure of a model though, we add on top of this accuracy, balanced accuracy, confusion matrices and processing time. This systematic procedure provides an objective evaluation of the strengths and practical weaknesses of each approach.

1.5 Scope of the Study

In this study, the authors do not perform physical quantum hardware experiments, but rather a simulation study of the algorithmic layer of surface-code decoding. We focus mainly on a distance-three (d=3) surface code measured over 3 rounds of syndrome measurement, with a physical error probability of 0.5%.Our software stack is intentionally streamlined, we use Stim for circuit generation and sampling, PyMatching for classical MWPM decoding and scikit-learn to create and test our machine learning pipelines. To cover this basic loop, additional experiments are performed in which the noise level is varied, different code distances are used and physical runtime performance is tracked.

Therefore the results and conclusions drawn from this work are limited to the simulated noise models, code structures, and algorithmic implementations used in this work. This study neither tries to claim that machine learning is always better than classical matchers, nor to set absolute fault-tolerance limits. Rather, it aims to offer a clear, regulated comparison that pinpoints precisely where machine learning is beneficial as a decoding helper, and the place it needs to be enhanced.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Quantum Errors and the Need for Error Correction

In quantum computing, information is encoded into quantum bits, or “qubits”, which are capable of existing in superposition states. This might be very expensive hardware, though. The interactions with the environment, imperfect control operations and measurement errors render qubits extremely fragile. Without correction, these errors can quickly add up and lead to loss of fidelity in quantum operations and make long calculations unpredictable.However, the noise in quantum systems is often quantized into three basic “Pauli” error types: the bit-flip (X), phase-flip (Z) and combined bit-and-phase flip (Y). Classical data can be easily duplicated and checked, but quantum states have the no-cloning theorem: an unknown state can't be duplicated and it can't be directly measured to check for errors without collapsing the superposition and destroying the data being saved.

This limitation can be elegantly avoided by distribution, through quantum error correction (QEC). Logical information is encoded in a network of entangled multiple physical qubits rather than one physical qubit. Carefully structured, non-destructive measurements can be performed on the system to obtain signatures of the noise without destroying the delicate logical state of the system. The data, then, is sent to a decoder which determines the most likely physical errors and corrects them accordingly.

2.2 Surface Codes

Many of the possible QEC strategies that have been suggested, surface codes are generally thought to be one of the most viable paths towards a fault-tolerant quantum computer. They have a major benefit: the geometry in which qubits are placed is 2D, meaning that they only need to interact with their nearest neighbours. Such spatial locality is well matched to the physical confines of prominent hardware architectures, including trapped ions and superconducting circuits.

A typical surface code architecture splits its physical qubits into two types: data qubits, the qubits used to store encodings, and measurement (or ancilla) qubits, the qubits used to extract noise signatures. Collective properties of these qubits, called stabilizers, are measured continuously to monitor changes to the system.

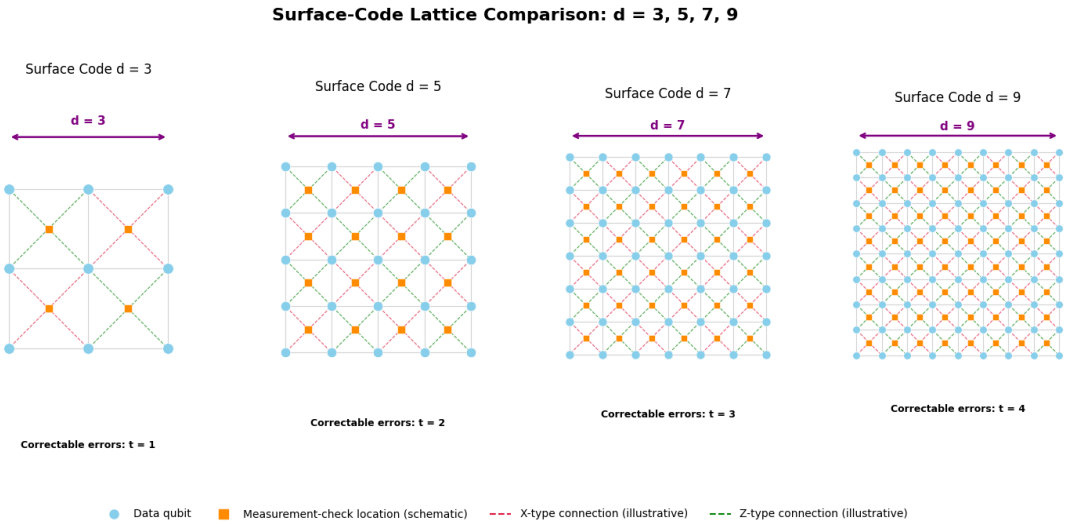


Figure 2:- *Surface code Lattice with distance (d=3,5,7 & 9) which illustrates the relationship between the code distance and the no.of correctable errors*

The surface-code lattice schemes for the various code distances are shown in Figure 1. The code distance (d) is the distance over which the minimum number of physical errors is needed to generate an undetectable logical error. The maximum possible error correcting ability for an odd-distance code is

$$t = \frac{d-1}{2}$$

So codes that have distances of 3, 5, 7, and 9 have the potential to fix 1, 2, 3, and 4 errors, respectively. Using a larger code distance will give better protection against logical errors, but will also need more physical qubits and more computational resources to decode the code. Errors of certain type. The code distance grown exponentially to reduce the logical error rate, but at the same time it demands a considerable amount of trade-off, mainly in terms of the amount of physical qubits needed, intensive measurement cycles, and classical computing power required to deal with the error data.

In the primary experiments of this project we consider a distance-3 (d=3) surface code that is performed over three rounds of syndrome measurements. This particular scale offers a computation friendly environment that is perfect for developing, training, and robust comparison of different decoding pipelines.

2.3 Syndrome Extraction and Logical Errors

The stabilizer measurement results from a quantum run are referred to as the error syndrome. Physical measurements on actual devices are also subject to noise, so stabilizers need to be measured several times on multiple rounds. Comparing the result of repeated rounds will help to identify changes in the measurement values. A detection event is defined as one of these changes.

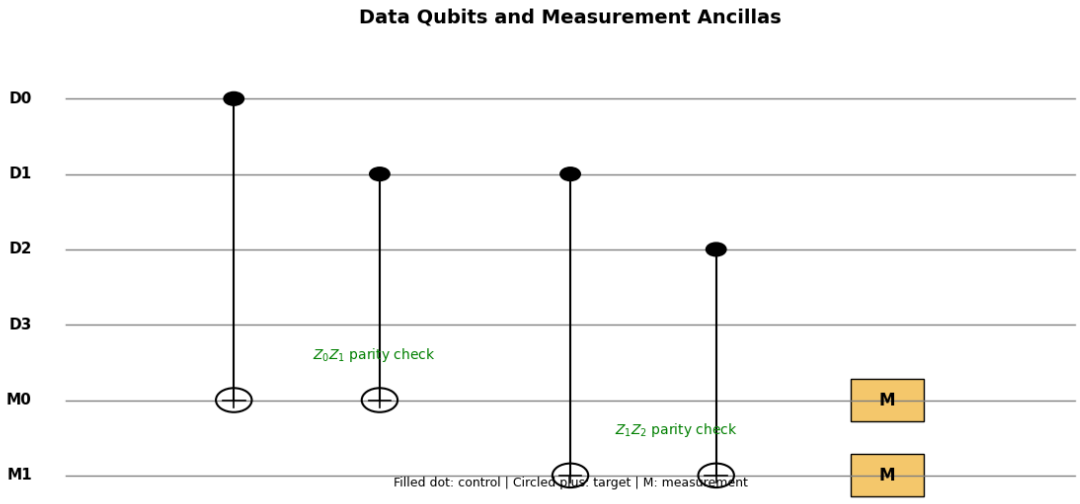


Figure 3:- *Simple circuit illustrating stabilizer measurement and syndrome extraction in the surface code*

The syndrome extraction is a process of interacting with the measurement ancillas and data qubits to get the measurement outcomes of the stabilizers. These results give information about whether there are any physical errors, but do not directly measure the encoded logical state. By repeating the stabiliser measurement, any changes in the measurement outcome can be recognized as detection events, which can then be used to infer the most likely error pattern by means of a decoder.

The decoder's task is to infer a likely error pattern from the observed detection events. A logical observable tracks whether the logical measurement outcome has flipped because of physical noise and the applied correction. If the decoder selects a correction that produces an incorrect logical outcome, the trial is counted as a logical failure.

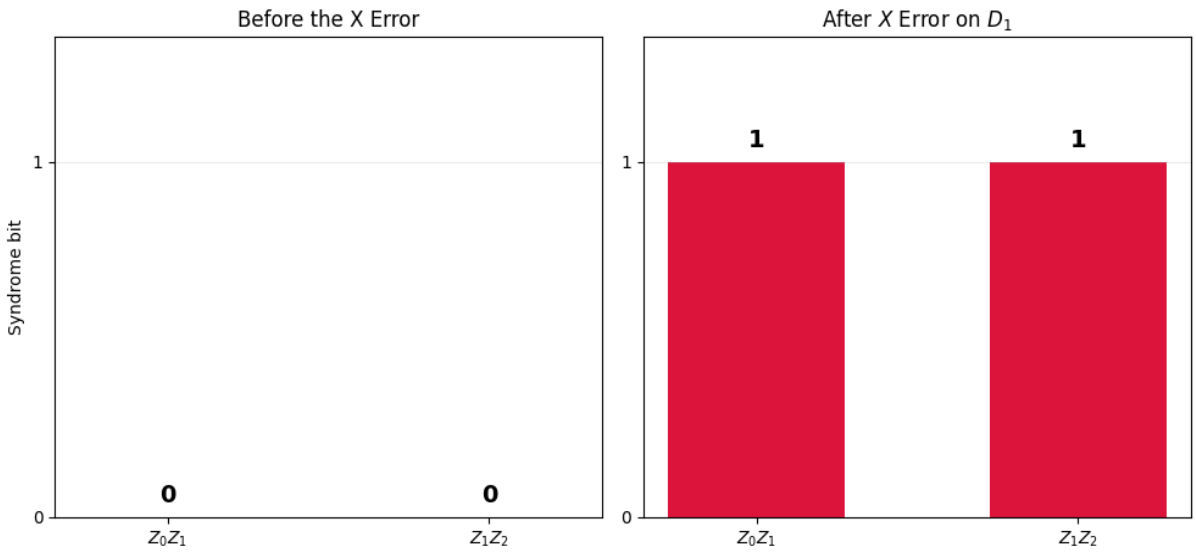


Figure 4:- *It illustrates how an error changes the measurement syndrome*

The above figure depicts an example of how an error on the data qubit can modify the syndrome pattern, by changing the result of one or more of the stabilizer measurements associated with the data qubit. An (X)-type error on a data qubit can flip the results of neighboring (Z)-type stabilizer checks due to anticommuting of the (X) and (Z) operators. The detection events give information on possible location and type of error. The syndrome, however, is not always uniquely sufficient to identify the physical failure so a good decoding strategy is necessary to decrease the logical failure rate.The success of a strategy for decoding words is measured in terms of logical failure rate, which is defined as:

$$P_L = \frac{N_{\text{Logical Failures}}}{N_{shots}}$$

At each shot, the number of incorrect decoding predictions is tallied as $N_{\text{Logical Failures}}$ and the total number of shots simulated is N_{shots}. Finally, minimising this failure rate when given a noise model is the main test for any decoder.

2.4 Minimum-Weight Perfect Matching (MWPM)

Minimum-Weight Perfect Matching (MWPM) is the traditional benchmark algorithm for surface code decoding. Solves the problem in a geometric way, by representing detection events as nodes on a graph. The nodes are the actual triggers detected, and the edges are the possible physical errors linking the nodes in the graph. The edges are assigned weights by the underlying noise model, with more rare errors being given more weight, and more common errors being given less weight.

The MWPM algorithm is a global algorithm that, given detection events, it attempts to find a set of paired events that minimizes the total weight of the connecting edges, as limited by the boundaries of the code layout. This creates a "perfect matching," which tells the correction path. One of our main baseline is MWPM, as it is mathematically solid and very optimized. By comparing our machine learning models directly against MWPM, we can determine if a data driven classifies can ever really beat MWPM in real life.

2.5 Machine Learning for Quantum Error Decoding

The data-driven decoders provide a novel approach to quantum error correction. A machine learning model, instead of using fixed mathematical algorithms for graphs, trains on simulated training examples to learn the subtle statistical relationship between detector patterns and logical outcomes. In effect, the model is able to find the underlying meaning of the noise from historical data directly. But it's not a silver bullet and requires adequate training data, a good architecture, specific noise profiles on which it has to learn, and careful hyperparameter tuning to be successful in practice.

In our setup, the measurements taken from the detector are used as the input features, while the binary target label is used to mark whether a logical-observable flip occurred or not. Since we want to have really good data integrity, we split our simulated data into three disjoint sets: the training set is used to fit the model parameters, the validation set is used to optimize hyperparameters and decision thresholds, and the test set is reserved solely for the final evaluation of the model.

To test this, we consider two different levels of machine learning models, and compare them directly to the classical MWPM baseline and check whether a data-driven classifier is able to provide reliable error suppression under the same experimental setting.

2.5.1 Logistic Regression

Our baseline “transparent” linear classification model is Logistic Regression. It takes all the features from the input detectors, performs a weighted linear combination of them and returns a probability of being between 0 and 1 via a sigmoid function.

This probability is then translated into a hard decision binary prediction using a selected decision threshold. Besides the standard logistic regression model, we train a class-balanced logistic regression model as well. This version deliberately increases the scale of the loss function in training to give higher importance to the classes that are under-represented.

This is especially important for quantum error datasets, as the logical flips generally do not happen as often as the non-flips. Class balancing can make the model much more sensitive to these rare flip events, but can also lead to an increase in the number of false positives. However, due to this trade-off, accuracy alone can't be used to make the whole picture; we need to evaluate it in terms of balanced accuracy, recall, precision and logical failure rate altogether.

2.5.2 Multi-Layer Perceptron

The Multi-Layer Perceptron (MLP) takes our machine learning methodology one step further by incorporating a feed-forward neural network that is able to learn complex, nonlinear relationships. The architecture designed for this project consists of an input layer corresponding to the features of our detector, two hidden layers with Rectified Linear Unit (ReLU) activation functions, and an output layer that estimates the probability of a logical-observable flip.

Hidden layers enable the network to identify complex spatial patterns of detector triggers, which cannot be described by a simple linear model. But the addition of architectural complexity always has two sides to the coin. A big network does not necessarily mean a good decoder; it can be seriously compromised by insufficient training data, highly imbalanced classes, overfitting and differences between the training and evaluation environments. Due to these risks, the MLP needs to establish its accuracy on totally disjoint test data sets, under the same evaluation setting.

2.6 Comparative Evaluation Framework

In order to create a highly stringent and consistent baseline, this study compared four different methods of decoding: conventional MWPM, standard Logistic Regression, class-balanced Logistic Regression, and the MLP. In this multiple-models comparison, we have an equal footing to examine the advantages and disadvantages of geometric and data-driven decoding.
The logical failure rate is the key indicator of success, defined as:

$$P_L = \frac{N_{Logical\ Failures}}{N_{shots}}$$

Since we need to have a finer-grained understanding of how these models classify the data, we add standard classification measures such as accuracy, balanced accuracy, recall and precision to the failure rate. We also calculate confidence intervals and perform parallel tests over independent random seeds when necessary, to ensure our results describe real trends in performance, rather than random samples.

To ensure the comparison is fair, all the decoders should be given the same circuit configurations, noise profiles, test samples, and target labels. Most importantly, the model selection and threshold tuning is done for the training and validation subsets alone. To avoid any accidental data leakage and optimistic optimization bias, the data is completely locked away in the test set until the final evaluation.
Finally, we also measure the execution times of the decoder to assess computational efficiency. We take these metrics with a grain of salt and make sure to take into account the underlying hardware, software optimizations, processing batch sizes, and the specific timing mechanisms we're using.

2.7 Scope and Limitations of the Approach

This project is an objective, simulation-based evaluation of decoding using machine learning in controlled conditions. We do not assume that machine learning is always going to be better than classical graph matchers. Rather, our experiments examine if logistic regression and MLPs can achieve competitive logical-error suppression and whether those performance gaps are consistent across multiple, randomized runs.

The results in this work can only be generalized to a distance-3 (d=3) surface code architecture. They can't be used on their own to set the trends of scaling to larger code distances, determine a clear threshold for fault-tolerance or guarantee success for completely different noise models. To answer those more general questions would need to be performing experiments over several code distances, with different physical error probabilities, and with different physical data environments.

The important point to note is that a decrease in the standard classification error doesn't necessarily mean that the logical failure rate will decrease. Similarly, if one model has a lower failure rate, then one may have to compromise on the processing speed. For these reasons, it is important to view accuracy, algorithmic reliability and computational cost as separate variables, which need to be considered separately.

2.8 Chapter Summary

The theoretical foundations provided in this chapter were laid out to assess conventional and machine-learning-based surface code decoders. We decided to use MWPM as our classical reference, and Logistic Regression and the MLP as candidates for the data-driven approach. The basis for our comparative framework is that testing environments have been identical, data is separated and performance is measured thoroughly. Finally, the usefulness of each of the decoding strategies will depend on the data and not on the claims of the particular strategy.

CHAPTER 3: LITERATURE REVIEW

3.1 Development of Surface-Code Decoding

Quantum error correction (QEC) has come from being a theoretical convenience to a near essential requirement for making quantum computers more reliable. However, real physical qubits are inherently temporary: to build a viable quantum computer, one must have a scalable architecture that identifies and corrects any physical errors in real time. In the past 20 years or so, surface codes have become a dominant and front-running contender in this area. That is, their prominence is mostly because of their 2-dimensional geometric layout, which permits for local interactions on the nearest neighbours, which is easily contained inside the strict layout constraints of modern quantum hardware.

Fowler et al. (2012) have given a thorough and clear roadmap for surface codes. Their work painstakingly detailed the stabiliser structures, the underlying error-correction mechanism and the high physical demands for large-scale fault tolerance. This framework set the basic language of contemporary studies on QEC.

However, a surface code is just as powerful as its decoding layer, even if the physical layout is optimal. This brings in the central computational problem of trying to understand a chaotic sequence of stabilizer measurement results and precisely infer the actual physical fault(s) that happened. This decoding step needs to occur repeatedly throughout a quantum computation, and hence the algorithm that is used determines the final logical reliability of the quantum machine, as well as the classical processing overhead needed to maintain it.

3.2 Minimum-Weight Perfect Matching

This classical solution solves the decoding problem from a geometrical point of view. The idea behind MWPM is to encode the raw detection events as nodes on a graph and the potential physical faults as weighted edges.The concept behind MWPM is to represent the raw detection events as nodes on a graph and the potential physical faults as weighted edges. It tries to find the lightest combination of defects that fits both the code layout and the given syndrome.

The elegance of MWPM is its mathematical clarity. In the case of a well-known underlying noise model, MWPM performs very well, and has a well-defined and interpretable decoding path. Its real-world performance however, is closely tied to the degree of accuracy with which the internal graph weights of it reflect real-world physical noise. The accuracy of the matcher can decrease if the errors are complex and correlated with each other and not as expected by the matcher. Moreover, the more code is scaled, the more important an engineering challenge the latency of the processing of MWPM becomes.

Even though there are challenges, the traditional decoding strategies have progressed in concert with the physical hardware. One impressive example was Google Quantum AI showing that logical error rates can be carefully reduced as the physical size of a surface code is expanded, as long as the physical qubits have a sufficiently high performance threshold. Recently, these limits have been extended, with a demonstration of below threshold surface code memories supported by real time decoding pipelines. These milestones are a reminder that decoding accuracy and execution speed must be studied together for any practical quantum computers.

3.3 Machine-Learning-Based Decoding

However, with the evolution of classical computing architectures, researchers have started considering machine learning as an alternative which is adaptive and data-driven. Supervised machine learning models seek to automatically learn the relationship between error syndromes and logical outcomes, without the need for human engineers to explicitly program the decoding graph, but by learning directly from many simulated or hardware-collected examples.

Initial studies indicated that the paradigm could be expanded to higher dimensional quantum codes using neural-network decoders, like in the works of Breuckmann and Ni (2018). Importantly, they pointed to a significant conceptually powerful feature of machine learning: the ability of a trained machine learning system to automatically learn from a asymmetric error distribution, which is complex and cannot be easily modeled by hand. To build on this, Varsamopoulos et al. (2020) explored distributed neural-network architectures that decode surface codes, and traced the challenging scalability issues that occur when training models for larger code architectures.

This preliminary work inspired research into a variety of network architectures and training protocols. The convolutional neural network (CNN) is one of the most prominent approaches studied by Bordoni and Giagu (2023), which is also well suited to detect spatial patterns in two-dimensional syndrome maps. The overall literature however, suggests that machine learning is not a panacea, and the predictive accuracy of the model rests on the exact fit of its design, training data, and test noise conditions.

It has progressed into ever more complex architectures. By 2024, researchers began implementing recurrent architectures and transformer-based decoders, and some works already investigated neural decoders on raw data directly obtained from experimental quantum processors. These recent advances are illustrative of the extraordinary progression of data-driven decoding. But they also identify a key challenge: It is not enough to extrapolate the performance of a large, highly specialized transformer model from the more advanced hardware and assume that it would replicate the same performance on a simpler linear model or generalized simulation workflow.

3.4 Comparison Conventional and Machine-Learning Approaches.

Comparing conventional graph-based decoders to machine learning models is like comparing apples and oranges. MWPM uses mathematically proven, global optimization. In the case of machine learning, it's done through the process of statistical estimation, establishing connections between inputs and outputs, and patterns that are programmed into its parameters throughout the trainings.

It is important to have a foundation in order to assess these paradigms. Logistic Regression is a simple, readily explained machine learning baseline. It provides an answer to the question: can a simple linear combination of detector inputs be used to predict a logical flip reliably? The baseline is improved by adding class balancing so that the model is forced to learn about rare logical flip events, which could otherwise be masked by the more abundant majority class. This, however, poses a tricky trade-off, as some models are prone to false positives if they are required to look for rare events.

The next step is a Multi-Layer Perceptron (MLP) which introduces nonlinear reasoning into the model with its hidden layers, enabling it to detect relationships among detector patterns that would otherwise be overlooked by a linear model. However, there's a price to this added power. To name

a few, MLPs are very sensitive to data quality, have a high tendency of overfitting and have notorious loss of accuracy when the evaluation noise model changes even minimally from the training data. In addition, a neural network is a fixed entity after it has been trained, and when the quantum code distance and/or the physical error rate change, it is usually necessary to undergo an entire training process with a new network.

These are significant structural differences, and to be fair, one must be absolutely consistent. Both paradigms need to be evaluated on the same independent sets of data in the same setting for classification performance and physical execution speed and compared separately.

3.5 Research Gap and Motivation of Current Study

The literature, however, has established that classical algorithms and machine learning models have a legitimate role in quantum decoding, but it is very challenging to draw broad conclusions from past studies. This is because the reported results differ significantly across teams, which employ varying code distances, circuit simulation styles, different noise models, and highly optimized neural network architectures.

This project aims to remedy this lack of standardization by developing a controlled entry level research benchmark. All of MWPM, standard Logistic Regression, class-balanced Logistic Regression as well as a Multi-Layer Perceptron are combined in a single, unyielding simulation pipeline. We consider the distance-3 (d=3) surface code with three rounds of measurements and assuming a fixed physical error rate of 0.5%. This removes external parameters and allows us to see how each model performs compared with the others. Data is strictly partitioned between training, validation and testing in our methodology. This avoids the issue of testing on data which was used to tune the model. Also, we make our final logical failure rate our ultimate metric of success and classify accuracy as a secondary indicator of success.

Finally, this study is unique in offering an honest comparison and baseline analysis without the premise that machine learning is superior. We instead allow the empirical evidence to reveal the models in which these specific models can compete effectively with MWPM, and those in which they have clear structural deficits.

3.6 Chapter Summary

There is a clear message in the existing literature: effectiveness of fault-tolerant quantum computers depends intimately on the efficiency of decoders. Machine learning provides an adaptable paradigm, whereas MWPM remains a strong mathematically sound reference point, and redefines the interpretation of error syndromes. This change, however, brings up great issues of scalability, noise dependency and computational latency.
Having this contextual backdrop in place, the next step is to outline our actual experimental mechanics to compare. The software tools, architectures of circuits, pipelines of datasets, and training protocols that underpin this research will be described in the following chapter.
The book introduces the reader to various computational tools and methodologies that are pertinent to this study.

Chapter 4: COMPUTATIONAL TOOLS & METHODOLOGY

4.1 Research Methodology

This work uses a simulation-based, data-driven approach to systematically explore machine-learning-assisted decoding for surface codes. The research pipeline spans six stages: simulation of quantum circuits, generation of raw syndrome data, dataset engineering, implementation of the decoder pipeline, model training/tuning and a thorough comparative evaluation.

To fully investigate this landscape, four different decoding frameworks are studied: conventional Minimum-Weight Perfect Matching (MWPM), standard Logistic Regression, class-balanced Logistic Regression, and a feed-forward Multi-Layer Perceptron (MLP). All four approaches are tested on the same, isolated test dataset to ensure the experimental integrity.

The main goal of our work is to understand how well each decoder is able to decode the raw error signals to predict which underlying logical-observable flip is responsible for each error. We have other classification metrics that are employed as secondary measures to decompose the behavior of our machine learning models, although the final logical failure rate is our ultimate indicator of engineering success.

4.2 Computational Tools

The entire experimental infrastructure is written in Python, exploiting a well-optimized scientific computing, quantum simulation and machine learning toolbox.

Tool	Purpose
Python	Central ecosystem and programming language
Stim	High-performance design and simulation of noisy quantum circuits
PyMatching	Fast graph-based implementation of the conventional MWPM decoder
Scikit-learn	Linear modeling, class balancing, feature scaling, and MLP execution
NumPy	Vectorized mathematical operations and multi-dimensional array management
Matplotlib	Data visualization and performance plotting

In this workflow, Stim does most of the work of quantum circuit generation and error sampling, and records the physical noise as localized triggers of the detectors. PyMatching automatically builds an optimized error-graph model of the underlying circuit for traditional decoding from the structure. Finally, scikit-learn is our data-science engine that powers everything from simple feature normalization to advanced training of neural networks.

4.3 Surface-Code Circuit Configuration

In our primary experiments we are using a rotated surface-code memory circuit, realized by Stim's built-in surface_code:rotated_memory_x template. The physical error probability of the simulator is set as 0.5 percent and the simulation is run over three consecutive rounds of syndrome measurement.

The complete parameters of our simulation environment are given below:

Parameters	Configuration
Surface-code configuration	Rotated X-memory circuit
Code distance	d=3
Syndrome-measurement rounds	3
Physical error probability	p = 0.005 (0.5%)
Physical qubits	26
Detectors	24
Logical observables	1
Circuit ticks	21
Total simulated shots	50,000
Random seed	42

The physical error probability is identical for all the fundamental circuit operations, thereby adding realistic noise channels like depolarisation of states, faulty resets of qubits and measurement uncertainty. Stim's noise generator processes these operations dynamically, determining the exact spatial locations and types of these operations.Each "shot" is a full cycle from initialization to final readout of the noisy circuit. We can take 50,000 different shots, and they will be statistically very large and diverse enough to test our decoders with a lot of rigor.

4.4 Dataset Generation and Preparation

For each simulated shot, two key pieces of data are generated: a sparse array of detector triggers, and a binary flag indicating whether the logical observable flipped. In our machine-learning pipelines, the signal detected by these detectors are directly used as input features, the logical flip state are our label for our classification.
On a mathematical level, our input feature space is described by an absolute binary matrix:

$$\mathbf{X} \in \{0,1\}^{N \times D}$$

Here N is the number of simulated shots (50,000) and (D) is the number of detector features (24). Accordingly, the vector to be targeted is:

$$y \in \{0,1\}^{N},$$

where the number 1 is a true logical-observable flip in our simulation convention and 0 means that the logical state was not corrupted.

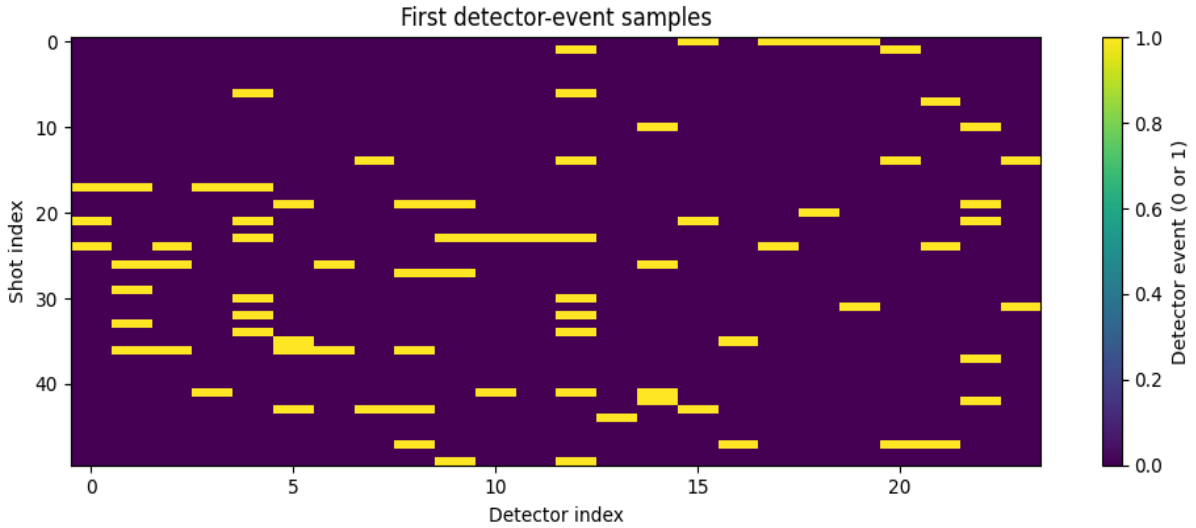


Figure 5:- *Heatmap Visualization of the Detector Event Samples Generated from Surface-Code Simulation*

The figure shows a heatmap of detector-event samples created for surface-code simulations. The shots are represented by rows and the detector indices by columns. Yellow areas are detector events (value 1) while dark-purple areas are areas where there was no detector event (value 0). The distribution of events shows the differences in measured syndromes between the various experimental shots. These detector patterns are used to feed the input features to the machine learning based decoder and to describe the error information that is available to decode. The decoder applies this syndrome information to predict the logical correction to be applied, and to assess the likelihood of logical failures.

The dataset contains 50,000 simulated shots: 44,810 were labelled class 0 and 5,190 were labelled class 1 (logical flip). This corresponds to an imbalanced label distribution. The dataset was split in a stratified 70:15:15 ratio using random_state=42, giving 35,000 training shots, 7,500 validation shots, and 7,500 test shots.

Subset	No.of.Shots	Purpose
Training	35,000	Model fitting and parameter estimation
Validation	7,500	Model selection, hyperparameter tuning, and threshold adjustment
Test	7,500	Final, un-biased decoder comparison
Total	50,000	-

The stratified split contains 35,000 training shots, 7,500 validation shots, and 7,500 test shots. The training set is used to fit model parameters, the validation set is used for model selection and hyperparameter tuning, and the test set is reserved for final evaluation. If class counts for each subset are reported, they should be calculated directly from the saved labels for each split; the total class counts must not be presented as subset counts.

4.5 Implementation of the Decoders

4.5.1 Minimum-Weight Perfect Matching

Our baseline is the traditional, graph-theoretic problem: Minimum-Weight Perfect Matching (MWPM). PyMatching directly imports the structural detector error model from the simulated Stim circuit, and builds an internal graph whose nodes are detector anomalies and whose edges are physical fault paths between these anomalies.

In the test mode, PyMatching operates on the raw detector arrays for each individual shot, and calculates the minimum-weight path between all the active nodes, from which it calculates the required logical correction. This correction is then compared with the actual target label, to determine whether the logical qubit was preserved by the matcher. MWPM is well optimized, mathematically sound, and gives a baseline for use in assessing our data-driven alternatives that is clear and transparent.

4.5.2 Standard Logistic Regression

This is the most basic and understood machine learning baseline, Standard Logistic Regression. The model is implemented in scikit-learn and trained with a set of linear weights per one of 24 features of the detectors, which represent the statistical probability of a logical-observable flip.
We use a StandardScaler pipeline to normalize the distributions of our input features, to make optimization stable. The fitted model gives a continuous score between 0 and 1, and a typical decision threshold is used to generate a binary prediction. This model gives us an exact measure

of the amount of decoding information that can be obtained with a purely linear classification model.

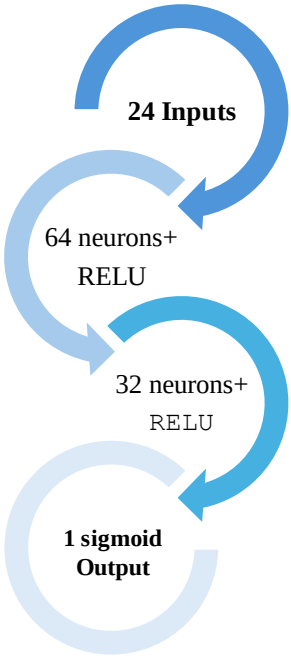
4.5.3 Class-Balanced Logistic Regression

We want to build a model that mimics the same linear pipeline as our standard model, but we need to make a crucial modification to our model to deal with our skewed data. Since only 11% of our samples are logical flips, the naive model is likely to overestimate its accuracy by always predicting "no flip".

To avoid this, the class-balanced variant scales the loss function of the internal model during the training phase, giving a much higher penalty if the model misclassifies the rare logical-flip events. This change makes the classifier more sensitive to miss errors, but may also result in a dramatic increase in false positives. Due to this shift in behaviour, we do not consider accuracy as a single classification metric for determining the usefulness of this behavioural change, but rather use a comprehensive set of classification metrics.

4.5.4 Multi-Layer Perceptron

We are adding nonlinear reasoning to our data-driven stack, with the Multi-Layer Perceptron (MLP) and the sklearn MLPClassifier. The 24 standardized features of the detectors are fed into the network and passed through a deep feed-forward network set up as:



These two hidden layers have 64 and 32 neurons, respectively, each of which is equipped with Rectified Linear Unit (ReLU) activation functions to capture complex combinations of detector triggers that are entirely beyond the reach of linear models, and occur in space. We train the network with the Adam optimizer and regularization factor α=0.001 for a maximum of 500 iterations. To get a pure baseline, early stopping is turned off and the internal seed is set to 42. For validation, we use the continuous probability output of the network to select the optimal decision threshold. We chose a decision threshold of about 0.55, as a fine cut-off using only trends in the validation. This architectural depth endows the MLP with great expressive power, but it

needs to be demonstrated on totally novel test data to warrant the substantial amount of computation required.

4.6 Performance Evaluation Metrics

The four decoding frameworks that we judge are on the basis of logical failure rates. To test the machine learning classifiers, however, a common suite of classification metrics are used in addition: accuracy, balanced accuracy, precision, and recall.

4.6.1 Logical Failure Rate

The absolute measure of the success of our decoding is the logical failure rate:

$$P_L = \frac{N_{fail}}{N_{shots}}$$

is the number of wrong logical corrections and N shot is the number of the tested samples. Finally, the decoder with the lowest logical failure rate offers the best protection of our logical information, under the noise model tested.

4.6.2 Accuracy and Balanced Accuracy

The overall accuracy is the percentage of correct predictions for the entire set of data:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Our dataset is quite heavily imbalanced so the accuracy metric can be very misleading. For this reason, we heavily rely on Balanced Accuracy which is the unweighted average of recall for both the majority and minority class and this gives the same score to the model's ability to detect rare logical flips.

4.6.3 Precision and Recall

Precision and recall give a fine-grained view into the performance of our data-driven models for classification problems. Mathematically, Precision measures our true positive rate of all our positive predictions:

$$Precision = \frac{TP}{TP + FP}$$

This measure is the percentage of logical flips that were also flips and can directly be used to measure the false alarm rate of the model. On the other hand, Recall (or sensitivity) is our true-positive rate within the total number of actual events:

$$Recall = \frac{TP}{TP + FN}$$

Recall is the percentage of correct logical flips that the model can correctly identify. This pair of metrics reveals the critical engineering compromise that arises in a data-driven approach to QEC: the risk of catastrophic error associated with failing to correct logical flips, and the cost in computation from correcting spurious flips.

4.6.4 Execution Time

In addition to the absolute accuracy, there is also the question of the computational resources needed to execute each decoding pipeline. Execution time is determined over a common set of tests on a consistent timing procedure, with a complete documentation of software dependencies and underlying hardware.

These timing results are deliberately not included in our logical failure rate analysis. Decoder speed and accuracy at suppressing errors are two entirely different dimensions of decoder performance – an extremely accurate model doesn't mean much if it cannot decode rapidly enough to match the real-time coherence windows of a physical qubit.

4.7 Experimental Consistency and Reproducibility

To keep things as fair as possible, we have a common circuit design, common noise characteristics and the same held-out test set for every decoder. No leakage of test-set information into the model optimization and tuning of the decision-boundaries.
We fix our random states to a seed value of 42 throughout the data splitting scripts and in the MLP architecture to create a highly reproducible pipeline. We note, however, that software seeds do not ensure the same numerical operations in different versions of the library or on different hardware.

It's also very important to document a limitation in our secondary workflow. The underlying notebook is a code-stress-test to load these decoders for various code distances and physical error rates, but there is an issue in the saved script in which the intended seed variable is accidentally overwritten during play. Due to this overwrite, this particular batch of data cannot be used as a valid, independent multi-seed evaluation.
For this reason, this chapter concentrates on our primary distance-3 experiment with 50,000 shots that was rigorously controlled. Other sweeping assertions about algorithmic scaling, noise generalization and certain absolute fault-tolerance limits are not intentionally made until corrected multi-distance experiments can be fully performed with independent seeding.

4.8 Chapter Summary

This chapter was devoted to describing the entire computational approach that was employed to compare MWPM, standard Logistic Regression, class-balanced Logistic Regression, and the MLP as a unified decoding ecosystem. We simulated a rotated distance-3 X-memory circuit using Stim, and were able to create a robust dataset of 50,000 independent shots with 24 discrete detector features and one logical observable.
With a 70/15/15 split, we neatly divided our data into training, validation, and testing pipelines. We adopted PyMatching's well optimised implementation of MWPM as our baseline and scikit-learn for building our data driven classifiers. Our evaluation framework is largely focused on the logical failure rate, bringing us up to speed and ready to examine how these models perform with the real data of errors. The empirical results of this face-off will be provided in the next chapter, with a discussion of the behavioural differences that were noticed amongst our four decoders.

CHAPTER 5: RESULTS AND ANALYSIS

5.1 Introduction

In this chapter, we compare four surface code decoding methods, namely, conventional Minimum-Weight Perfect Matching (MWPM), Logistic Regression (LR), class-balanced Logistic Regression (LR-CB), and the Multi-Layer Perceptron (MLP) in an empirical manner by making an exhaustive head-to-head comparison. For full experimental fairness, all the decoders were tested using a test set that was seen in exactly the same way, and that all differences in their performance are due solely to differences in their decoding logic.

The ultimate measure of success is the logical failure rate, which is the true percentage of test cycles that a decoder fails to correctly infer the state of the logical qubit. The results presented here have been taken from a standard simulated architecture with code distance 3 & 3 consecutive rounds of syndrome measurements and a baseline physical noise floor of 0.5%.

5.2 Experimental Configuration

The basis of our comparative evaluation is 50,000 independent shots simulated using Stim. This raw data was partitioned into isolated training, validation, and testing sets using a very precise partition of 70/15/15 and stratified.
The final experimental run is described in detail below with a focus on the structural parameters that are used to control this run.

Parameter	Configuration
surface code	Rotated X-memory surface code
Code distance	3
Syndrome measurement rounds	3
Physical error probability	0.5%
Total Simulated	50,000
Number of Detectors	24
Number of logical observations	1
Training Samples	35,000
Validation Samples	7,500
Test Samples	7,500
Random seeds	42
Primary Evaluation Metric	Logic Failure rate

Each decoder was tested with the same 7,500 shots, under the same conditions. The test partition was highly skewed towards the class 0 (unperturbed logical observable) with 6722 samples and class 1 (true logical flip observed) with 778 samples. Importantly, the MLP's final classification threshold was optimized using only the validation data and the test set was left completely uncorrupted during the optimization process.

5.3 Benchmarking Decoder Performance

The big discrepancies in performance are apparent when we feed our test data (which we withheld from the training process) into the four pipelines. The number of decoding trials for these trials are recorded.
The Below table compares the logical failures for the four decoding methods. The following table shows a comparison of log failures for the 4 decoding methods.

Decoder	Logical failures	Test Shots	Logical failure Rate
MWPM	144	7,500	1.920%
Standard Logistic Regression	807	7,500	10.760%
Class-balanced Logistic Regression	1,558	7,500	20.773%
Multi-Layer Perceptron (MLP)	186	7,500	2.480%

To provide a visual comparison of the four decoding methods, the logical failure rates obtained from the common test set are presented in Figure 5.2. The comparison includes MWPM, standard Logistic Regression, class-balanced Logistic Regression and the Multi-Layer Perceptron (MLP). All models were evaluated using the same 7,500 test samples at code distance (d=3) and physical error probability (p=0.5%)

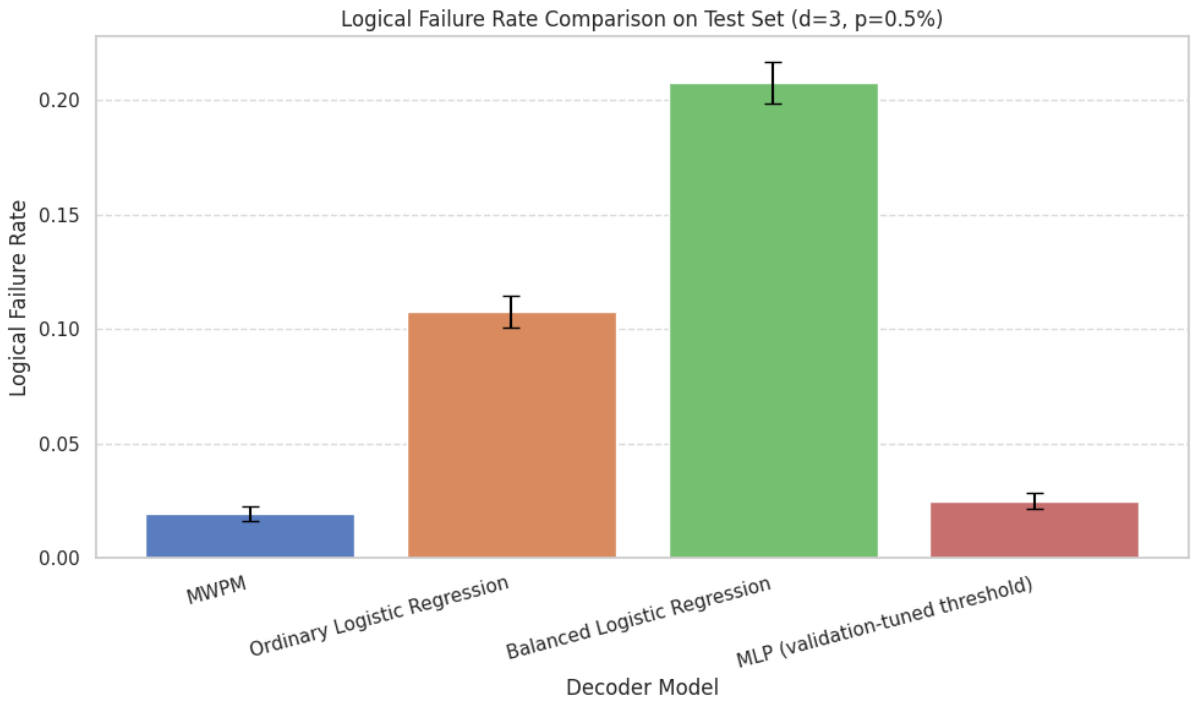


Figure 6: Comparison of test-set logical failure rates for MWPM, standard Logistic Regression, class-balanced Logistic Regression and MLP at (d=3) and (p=0.5%).

As shown in Figure 5, MWPM achieved the lowest logical failure rate of 1.920%, followed by MLP at 2.480%. Standard Logistic Regression and class-balanced Logistic Regression recorded higher failure rates of 10.760% and 20.773%, respectively. These results indicate that the MLP performed better than the two Logistic Regression models but did not outperform the MWPM baseline in this experiment.
The final logical failure rate is:

$$P_L = \frac{N_{\text{Failures}}}{N_{Test\ shots}} \times 100\%$$

In our case the equation gives us a very good result:

$$P_L = \frac{144}{7{,}500} \times 100\% = 1.920\%$$

The facts confirm the order. Overall, MWPM had the best error suppression, which resulted in the lowest logical failure rate. The deep MLP was an excellent runner-up, while the both linear logistic regressions had very high failure rates.

5.4 Analysis Of MWPM Decoder

As our standard model, MWPM is based on global geometric optimization, with a weighted matching formulation which is directly based on the underlying detector error model of our circuit. MWPM had 144 failures for 7,500 test shots and had a logical failure rate of 1.920%. This is a great reference that demonstrates the need for assessing data-based models with known classical matchers. Being judged solely by the internal classification statistics of a neural network is dangerous; it has to be shown to be useful in comparison with a simple error suppression power of the graph theoretical baselines.

To examine the response of the MWPM decoder to different levels of physical noise, its logical failure rate was evaluated across multiple physical error probabilities. The resulting trend is presented in the figure .

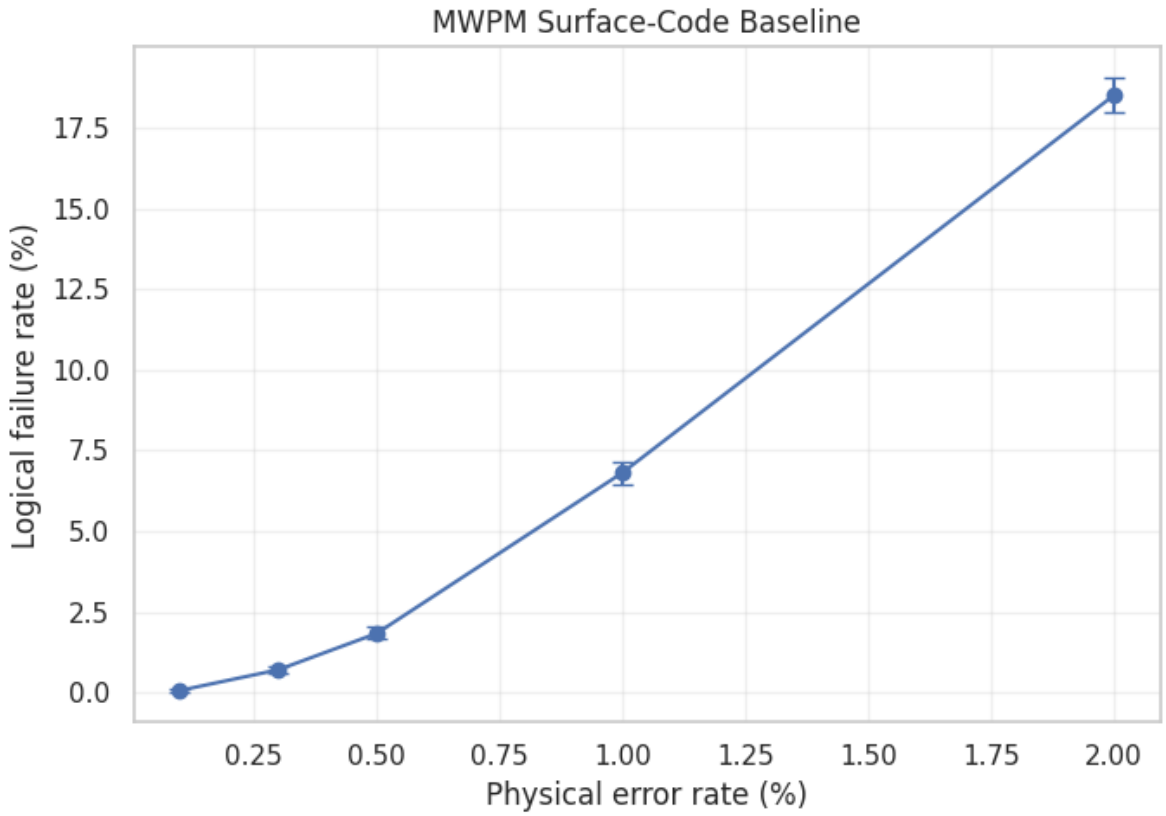


Figure 6: *Logical failure rate of MWPM decoder as a function of physical error probability.*

The Figure shows an increase in the logical failure rate as the physical error probability increases. This behaviour indicates that higher physical noise makes reliable logical decoding more difficult. The results provide a useful baseline for the comparison with machine-learning-based decoders.

5.5 Logistic Regression Models

5.5.1 Standard Logistic Regression

The standard Logistic Regression model had 807 logical failures, which resulted in a very high rate of logical failures of 10.760%. This is a huge drop from the classical baseline performance. Logistic regression provides a neat, readily interpretable baseline estimate of the probabilities, as it is modeled linearly as a function of the syndrome features, but the structure of logistic regression becomes quite apparent here. The simple linear decision boundary is fundamentally insufficient for capturing the highly correlated and complex topological networks that are formed by stabilizer detection events. Under this configuration the model is not as effective as MWPM and the MLP as it fails to make the deeper contextual relationships in the syndrome data.

5.5.2 Class-Balanced Logistic Regression

The class-balanced Logistic Regression model produced 1,558 logical failures among the 7,500 test shots, corresponding to a logical failure rate of 20.773%. This result shows that class weighting did not improve the final logical-failure rate in this experiment. The explanation should be treated as an interpretation, not as a demonstrated causal mechanism.

This poor performance is a significant warning to watch for dataset engineering. Although class balancing is supposed to not be drowned by a few minority classes, it does not necessarily result in a better decoding logic. The model had been actively trying to maximize the representation of rare logical shifts by warping the internal loss function. This shift led to a huge surge of false predictions. Class balancing in this way turned out to be counterproductive with regard to minimizing the number of logical failures per class for our objective.

5.6 Analysis of MLP Decoder

We used standardized input features with a two-hidden-layer MLPClassifier from scikit-learn. The reported architecture uses hidden layers of 64 and 32 neurons, respectively, with ReLU activation, the Adam optimizer, and an L2 regularization coefficient of 0.001. Confirm the layer order against the final code before submission.
The MLP had 186 logical failures on the 7,500 test shots for a very competitive logical failure rate of 2.480%.

This is an impressive achievement for our automated line of production! The hidden layers of the MLP were able to learn very nonlinear decision regions, reducing the logical failure rate from 10.760% to 2.480% with the linear model. The network, however, did not achieve the same as the classical baseline, with a score of 0.560 percentage points behind MWPM in this impressive network's ability to learn complex syndrome patterns. Therefore, the performance of the MLP must be considered as a promising proof-of-concept for data-driven decoding, but not as algorithms being superior.

5.7 Comparison Between MWPM and MLP directly.

Since the MLP turned out to be our strongest machine learning candidate, we can compare the performance of the MLP directly to our gold-standard classical matcher, which enables us to pinpoint the specific statistical subtleties between the two paradigms.

Controlled Variable	Experimental Outcome
Average MWPM logical failure rate	1.920%
The logical failure rate for MLP	2.480%
Absolute difference	0.560% points
The relative increase in MLP failure rate compared to MWPM	29.2%

The 0.560 percentage point increase seems rather insignificant, but it amounts to a significant 29.2% increase in logical failures for the neural network.

A paired analysis of their individual predictions on the same test shots was conducted to gain deeper insight into this behavior. This showed an interesting intersection: there were 56 cases where MWPM was unsuccessful, while the MLP was successful in fixing the mistake, and 98 cases where the MLP was unsuccessful, but MWPM succeeded. We conducted an exact paired-binomial test to see whether this difference is statistically significant or not; it came back with an approximate (p)-value of 0.000894. This result is well below the significance levels typically used in statistics, so the results were statistically significant, and indicated that MWPM was strongly favored.

5.8 Reproducibility of Independent Random Seeds.

The experiments were repeated five times with five different random seeds and with the same simulation setup to check the consistency of the results of the decoder. The logical failure rates obtained in the individual runs were compared to see if the relative performance of MWPM and the MLP was the same in all runs. The distribution of the failure rates recorded for both the decoders is shown in below Figure.

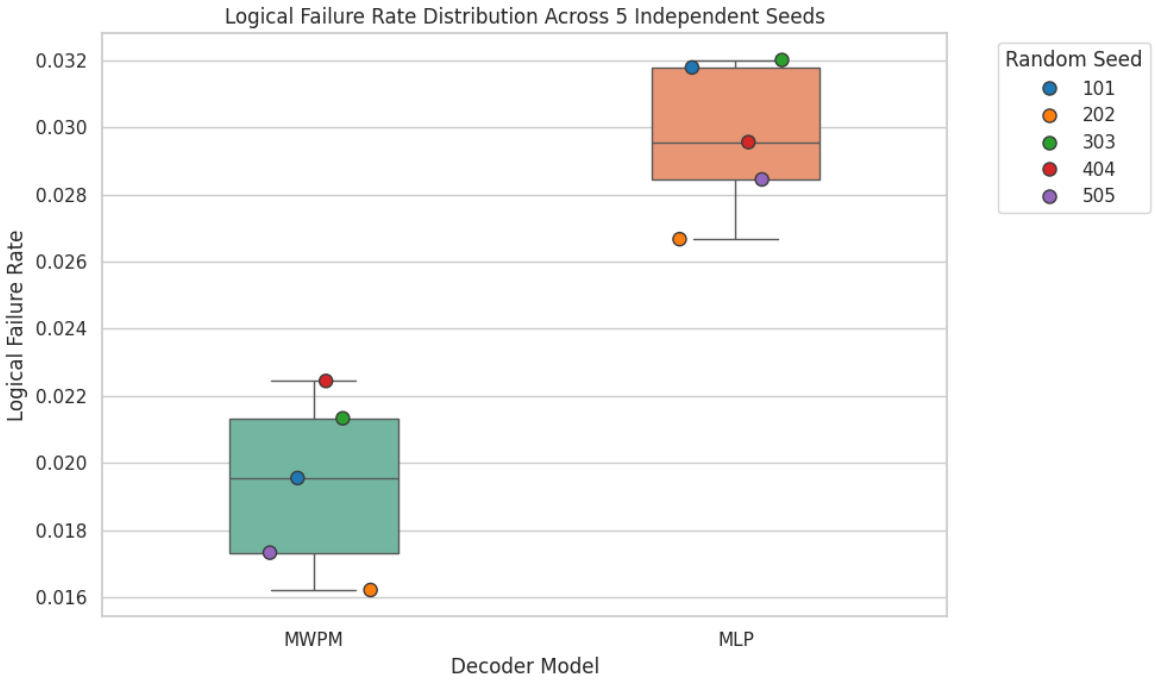


Figure 7: *Logical failure rate distribution for MWPM and MLP across five independent random seeds*

From the results, it can be observed that the logical failure rates of MWPM were lower compared to the MLP in five runs. This consistency implies the difference observed wasn't just confined to one random seed in the conditions being tested. The result, however, needs to be analyzed in the context of the chosen code distance, the physical error rate and configuration of the model

5.9 Benchmarking Of Various Code Distances:

The code distance is not the only measure that should be considered when the performance of a quantum error-corrector decoder is measured, but also as the dimension of a code grows. The logical failure rates of MWPM and the MLP were studied for the code distances and physical error rates that were used in the experiment for the scaling. This comparison gives a preliminary idea on the performance of the two decoding schemes in various simulation scenarios. These results will be displayed in the following figure

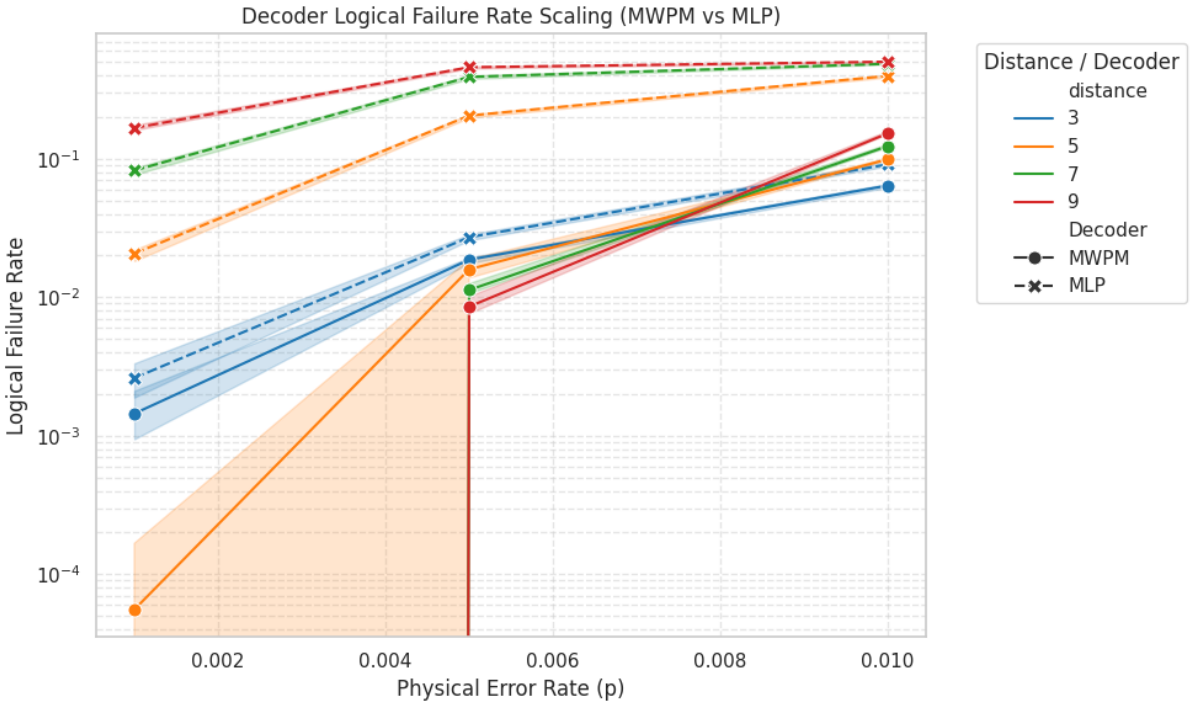


Figure 8:- *Comparison of Logical Failure rates for MWPM and MLP inference*

The results of the various configurations evaluated are shown in figure. Both code distance and physical error probabilities needs to be taken into consideration in interpreting the results as they affect the decoding task. Final results on scaling or generalisation should only be based on successful running on independent seeds and on the same experimental conditions. These results should particularly not be considered as an indication of a fault-tolerance threshold without conducting a proper threshold analysis

5.10 Testing Computational Performance & Decoding Speed of algorithms

When comparing quantum error-correction decoders, computational performance is another factor to take into account, in addition to the logical failure rate. A low logical failure rate decoder might not be as useful if it takes a longer time to decode the syndrome data. Thus, the decoding latency and throughput for MWPM and MLP were evaluated in the above-mentioned benchmarking

scenario. They are compared with respect to their measured computational performance in Figure 9.

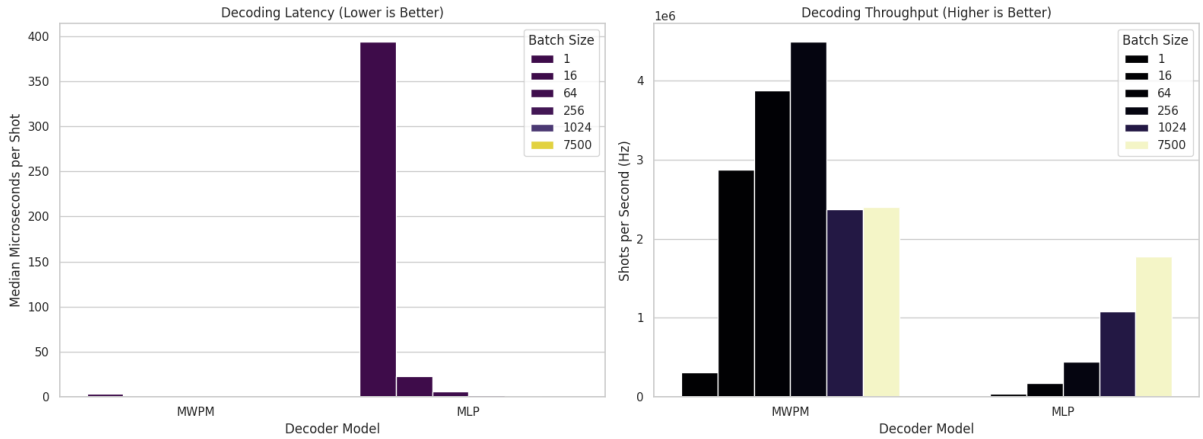


Figure 9: *Comparison of throughput and decoding latency for MWPM and MLP inference.*

The results of the benchmark show that in the tested environment, the median decoding latency of MWPM is lower than that of MLP and the throughput of MWPM is higher than MLP. Therefore, in this experiment, the MLP did not show a speed improvement. These measurements are dependent on the hardware, software implementation, batch size and timing procedure. They should therefore be viewed as one benchmarking and not as a ranking of the two decoding methods.

5.11 Overall Discussion

On a birds-eye view of the entire spectrum of our results three basic understandings come to the fore:

Absolute reference point

- MWPM is the absolute benchmark: With a logical failure rate of 1.920%, this baseline was clearly set by the classical ‘graph-matching' algorithm, that machine learning configurations could not match.
- nonlinear modeling is necessary for data-driven QEC: The results of the MLP are much better than those of the two linear logistic regression models, indicating that the decoding is nonlinear; Such complex spatial correlations require deep, highly expressive models that need to be the focus of future machine learning architectures.
- Naive training tweaks can go wrong: conventional data-science strategies may not work and the class-balanced model failed to perform well in this case. In QEC, an even model over its class predictions is basically worse than a model that tries to minimize the absolute number of logical failures.

5.12 Limitations of the Current Results

Our empirical results give a clear ranking of the decoders, but should be understood in the exact context of our experimental setup. The limitations do not invalidate our comparative analysis, but actually constrain the scope of our analysis and conclusions and help us identify the next steps for this research:

- A single primary random seed is used: The main comparison is based solely on data from a random seed value of 42. To ensure absolute stability of the performance trends observed, parallel experiments at multiple independent random seeds are needed.

- Small code distance: We only consider a distance-3 (d=3) configuration in our experiments. Therefore, the scaling of these machine learning models and their ability to reduce the error gaps from MWPM at larger topological scales is not predictable from this data.
- Physical noise floor: Four decoder face-off was done only at a physical noise floor of 0.5%. To test the decoders in different noise conditions, a wide range of physical error rates needs to be covered.
- There is no clear threshold: Current data make it difficult to estimate a true threshold for fault tolerance. To gain a definite threshold calculation, the evidence of crossing points should be clear and with simultaneous scaling of both code distance and physical error rate.
- We don't have any speed advantage that's established at this time: execution latency is a major concern for real-time decoding, but at this time our current framework doesn't provide a reliable timing baseline. To make a rigorous performance comparison, one must ensure the processing batches are standardized, the hardware is dedicated for a specific profile, the system is warmed up according to definition and repeated measurements are averaged.
- Model and threshold dependence: The reported performance is very dependent on our hyperparameter choices, such as our StandardScaler pipeline, our Adam optimizer parameters, and the classification threshold we used from the validation set (0.55). The behavior of the classifiers might noticeably change if any of these design variables are changed.

5.13 Chapter Summary

This chapter provided a comprehensive empirical comparison between MWPM, standard Logistic Regression, class-balanced Logistic Regression and the MLP, each tested with a common (and unbiased) test set of 7,500 simulated shots. Conventional MWPM had the best error suppression performance, with the least logical failures rate of 1.920. Standard Logistic Regression and class-balanced Logistic Regression were the obvious bottom two, with scores of 10.760% and 20.773%, respectively.The scores of 10.760% for standard Logistic Regression and 20.773% for class-balanced Logistic Regression were the obvious bottom two.

Finally, these results demonstrate that the MLP provides a significant step over simple linear classifiers, but falls short of the best graph-theoretic global classifiers at short code distances. This analysis lays the groundwork for a fully reproducible baseline for data-driven quantum error correction. In order to develop this baseline into more general statements of algorithmic generalization, fault-tolerance scaling or real-time processing efficiency, our future workflows need to focus on independent multi-seed validation, large code-distance scaling, and broad physical noise sweeps.

CHAPTER 6: DISCUSSION

6.1 Overview of the findings.

This work aims to examine the possibility of using machine-learning assisted decoding for surface codes by comparing three classifiers which are data-driven in a strict head-to-head comparison with a conventional decoding method, namely the Minimum-Weight Perfect Matching (MWPM) decoding. Instead of building on architectural speculations, we focused on a realistic problem: Can these various forms of computation, when studying outcomes of a local detector, accurately model the underlying logical-observable defect occurring in a single simulation.

For comparison, a distance-3 (d=3) rotated surface-code memory circuit has been compared with a baseline physical noise floor of 0.5%, in three rounds of repetitive syndrome measurements. From this architecture, we designed a complete dataset of 50,000 independent shots using Stim, which enabled us to separate the dataset into a stratified training, validation, and testing pipeline. We were able to check the results of the four held-out decoders on a shared set of test data to prevent any data discrepancies from influencing the results of our performance metrics.

Through our empirical results, we uncover apparent and measurable differences in error suppression. The conventional MWPM had the greatest fidelity with a logical failure rate of 1.92%. The feed-forward Multilayer Perceptron (MLP), one of the data-driven candidates, was a close runner-up with failure rate of 2.48%. In contrast, the failure rate for standard Logistic Regression and class-balanced Logistic Regression was extremely high, of 10.76% and 20.77% respectively.

These key findings lead to two obvious takeaways. First, they demonstrate that the nonlinear learning ability of the MLP can represent important error structures which linearly based models fail to represent entirely. Second, they show that in this short-distance regime it is still necessary to beat the performance of highly optimized classical algorithms such as MWPM. Finally, the thesis sets out a clear performance threshold and identifies specific regions that still need to be developed in greater detail using data-driven architectures.

6.2 Benchmarking of Decoders

By comparing the stark differences in performance between each decoder, it is easier to understand how the mathematical architecture and learning ability of each decoder affect their performance in practice. The specialized design is the key to MWPM's highest performance. It maps the raw detection events onto a global graph structure by using the specific edge weights that are optimized for the problem and the detector error model of a given circuit. It solves the error correction as a structured geometric optimization problem, enabling efficient pairing of defect chains. This focused mathematical foundation is the reason MWPM had the lowest logical failure rate, 1.92%.

In contrast, standard Logistic Regression had a logical failure rate of 10.76%. Its decision boundary is defined by a linear combination of detector features, which limits its ability to represent nonlinear relationships between detection events. A physical fault can produce correlated detector events across space and measurement rounds.
Class balancing worsened this linear baseline further, and caused a massive increase in logical failures up to 20.77%. Class weighting is a common tool in data science to help models find rare minority events, but when applied in quantum error correction it can be extremely counterproductive. The model was able to heavily punish minority-class errors in the fitting stage, causing it to make a huge number of false positive predictions. The result shows one of the key

design principles in quantum information research: It is not optimal to maximize a model's class predictions but to evenly distribute those predictions.

These linear constraints were avoided by the nonlinear MLP, obtaining a competitive 2.48% failure rate. The use of a hidden layer of 64 neurons and another of 32 neurons in a feed-forward network enabled the network to learn the complex spatial combinations of error syndromes, as it was fed the 24 detector features. This deep configuration brought a huge improvement over our linear ones, but the final error profile was only slightly wider than our graph-theoretic reference.

This can be seen as a single summary of the final face-off of multi-decoders:

Decoder	Logical Failures	Logical Failure Rate
MWPM	144/7,500	1.92%
Standard Logistic Regression	807/7,500	10.76%
Class-balanced Logistic Regression	1,558/7,500	20.77%
Multi-Layer Perceptron (MLP)	186/7,500	2.48%

The MLP reduced the logical failure rate by approximately 77% relative to standard Logistic Regression and 88% relative to class-balanced Logistic Regression. Compared with MWPM, however, the MLP had a 29.2% higher relative logical failure rate in this test set. These comparisons describe the tested configuration and do not establish universal scaling behaviour.

6.3 Interpretation results from the Machine Learning Models

One of the most interesting findings of this work is that network complexity is not the only determining factor for a machine-learning decoder's success. Our findings also indicate that hidden layers are not a sure bet for a model to gain a competitive advantage over conventional algorithms; however, it demonstrates that nonlinear modeling is indeed necessary.
To fill the performance gap between traditional matchers and neural networks, we need to break down some important factors:

First, MWPM uses structural information encoded in the detector error model, which supplies graph edges and weights derived from the circuit and noise model. The MLP instead learns statistical relationships from labelled training data. This difference in how the methods use prior information is one plausible reason for the observed performance gap, but the present experiment does not isolate its causal contribution.

Second, dataset size and the rarity of some error patterns may affect the MLP's performance. The 50,000-shot dataset provides a baseline for this distance-3 experiment, but it does not establish whether the dataset is sufficient for rarer patterns or larger code distances. Those questions require additional experiments.

Third, the training constraints and threshold tuning have a strong impact on our final results. For the sake of preserving the integrity of our workflow, we optimized the MLP's decision boundary only in our validation split, which gives us a threshold of 0.55. This is a total solution to the issue of optimistic evaluation bias, but a static threshold computed using the data from a set of validation samples will not necessarily be perfectly balanced across all cases it will be exposed to in a new test partition.

Lastly, the exact setup of our feature vectors, hidden layer nodes and Adam optimization parameters plays a big role in the performance of our network. Anyone of these design levers could

noticeably change the error profiles of the classifier. We see these architectural details not as a failing, but as a great opportunity to optimize them in the future, and value them very highly.

6.4 Research Significance and Contribution

The major contribution of this thesis is the controlled and unbiased computational comparisons of conventional and data-driven surface-code decoders. This work does not presume that machine learning is a complete replacement for the traditional system, but rather tests all candidates against a similar environment, in which they have to meet with a very specialized problem baseline.

Our research makes three unique contributions to the field:

→ First, it provides a fully reproducible experimental framework. We carefully define a step-by-step simulation flow that starts from a noisy circuit and ends with feature engineering of the raw syndrome, data isolation, and multi-model benchmarking. This clean pipeline is a stepping stone for future researchers to apply more complex models under more stringent conditions.

→ Second, it explains the line between linear and nonlinear learning. This study demonstrates that surface-code decoding is an inherently nonlinear task by demonstrating that the MLP was significantly better than logistic regression in both variants. At the same time, it reveals that the raw learning capacity of classical matchers is insufficient to complete the beat of highly optimized classical matchers without considering domain-specific structural constraints.

→ Third, it specifies precisely the following research questions that need to be answered. Our data illuminates the exact nature of the restrictions of the entry-level classifiers and brings the focus to critical questions about code-distance scaling, sweep of physical errors, multi-seed stability and processing latency. These lines of investigation will be aimed at identifying whether advantages other than simple accuracy can be gained by using machine learning, for example, real-time adaptability or a fast pre-decoding assistant, where the machine learning method is used in conjunction with a traditional matcher.

In conclusion, this work does not suggest that the machine learning approach has surpassed the traditional decoding approach. Its real power is in creating a measurable, real-world benchmark, revealing the subtleties of a naive balancing of data sets, and defining a direction of action for data-driven error correction.

6.5 Methodological Strengths

A comparative analysis is supported by a number of intentional methods. Our full evaluation is based on the same test set that is completely separate from the training set, so there is no risk of the test set being different in difficulty from the training set. All decoder error syndromes and reference labels were identical, so any differences in performance were a result of the decoding logic only.

Additionally, our rigorous data isolation firewall between training, validation and testing data removes any possibility for optimization bias and data leakage. To optimize the decision threshold of the MLP only on our validation partition and not on the test-set, we will ensure that our obtained metrics are truly representative and accurate of the performance of these decoders on unseen data. Traditional architectures combined with data-driven architectures offer a very powerful range of baselines. Standard Logistic Regression provides a pure and simple linear baseline, while its class-balanced variant allows us to uncover the performance of models that have been compelled to

over-index for minority events. The Multi-Layer Perceptron brings in an important element of nonlinear reasoning, and we take MWPM as our absolute graph-theoretic baseline.

In collaboration, these decisions have created a very strong back-bone for systematic decoder benchmarking in our experiment. But it should not be confused for complete proof of architectural generalization or absolute fault tolerant performance as the methodological strength of this comparative study is not to be interpreted as such. First, a number of great challenges have to be tackled through further experimentation to unlock those more general conclusions.

6.6 Limitations of the Present Study.

In order to interpret our findings with proper academic honesty, we need to assess them in the context of the experiment we implemented. These are the exact limits of the conclusions we reach now:

- Limited code distance where We only use a distance-3 (d=3) surface code in our core benchmark. The behaviour of decoder error rates or the training dynamics cannot be determined by examining a single static layout scale.
- Physical noise environment was restricted to 0.5%. Only at this physical noise level we could compare the four decoders. These performance differences might be strikingly different under other error probabilities and/or hardware noise models, as algorithmic behaviors are sensitive to changes in the levels of noise.
- Limited exploration of architecture in this Only two linear regression models and one MLP layout were explored for this project. In our pipeline, we never actually test any massively sized family of more complex architectures, like convolutional networks, transformers, or custom-made loss objectives.
- Our main experiments are conducted with a single random seed that is kept locked. This design decision allows us to have consistent results for our specific run, but does not allow us to follow the natural statistical deviation that occurs when we vary the dataset and model initialization.
- No proof of fault threshold This work does not try to calculate the fault threshold of the surface code. To obtain a true mathematical threshold, scaling sweeps must be conducted simultaneously on code distance and code floor with a completely standardized test procedure, and be across multiple variables.
- No empirical processing baseline: We currently have accuracy measures of error-suppression, but not measures of processing speed. The MLP is not necessarily faster at making logical corrections than MWPM; a well-controlled, hardware-segregated latency test for the MLP has not yet been performed.
- Strictly simulation-based data: All our workflow is based on idealized, circuit-level simulations. The mathematical formulation of this research would need to be widely tuned to adapt to the complex, noisy noise environment and non-uniformity of error channels found on a physical quantum processor, before such an implementation could be considered for actual deployment.

These limitations do not detract from the results of this study; they actually form a very clear, specific set of goals for future research.

6.7 Future Research Directions

Our empirical observations directly drive the next generation of this decoding pipeline, where the evaluation and structurally improvement of the data-driven classifiers is a priority for scaling. We wish to suggest seven main lines of research:

1. ***Scalability to larger topologies****:* In future iterations, a number of code distances (such as (d=5, 7)) need to be tested simultaneously, and the circuit template, noise mechanism and test metrics should be kept fixed. This is going to show you exactly what error-suppression curves will look like as the system gets bigger and bigger, as the model gets closer and closer to reality.
2. ***Systematic Noise rate Evaluation***: The decoders should be set up to compete against each other over a wide range of physical error probabilities. Reporting these results to strict uncertainty intervals will demonstrate the trends to be expected in terms of fault tolerance and will reveal if data-based error suppression is in line with the expected trend.
3. ***Multi-seed independent validation****:* We need to repeat the whole pipeline for generating the data and optimising the model over many independent random seeds. Our results will be based on real statistical measures—means, variance, and confidence intervals—that will ensure that our conclusions represent trends and not statistical anomalies.
4. ***Improved Neural Network****:* Future pipelines should consider alternative network designs, such as Convolutional Neural Networks (CNNs) to interpret the 2D syndrome maps; feature representations; dropout regularisation; and custom QEC loss functions.
5. ***Systematic generalization testing****:* It is, monitoring of the performance of the model when it is thrown into a completely new environment when the noise floor or code distance is different from the one used during training. The experimental design should clearly draw lines between standard interpolation, out-of-distribution shifts, and true structural transfer to larger code scales.
6. ***Controlled runtime benchmarking***: This latency study is a critical aspect of hardware-isolated runtime benchmarking. Decoding processing time needs to be measured on the same hardware, excluding the overhead of data formatting and feature pipeline prep, while keeping only the pure model inference latency.
7. ***Hybrid Decoding Pipelines***: The exploration of hybrid decoding pipelines: Rather than relying on a neural network to function as a complete decoder, it may serve as a fast assistant, for example, by recognizing very challenging detector clusters or by predicting the dynamic edge weights to optimize a primary classical matcher.

Future studies can clearly establish the scalability, processing efficiency and real-world applicability of machine-learning based error correction by pursuing these specific directions.

6.8 Chapter Summary

In this chapter, I provided a detailed description of our four-decoder benchmark and its implications for research. The error suppression performance of conventional MWPM was the highest at 1.92%, while the nonlinear MLP was competitive at 2.48%. The MLP is an empirical demonstration of the need for highly expressive, nonlinear reasoning to decode the surface code, as it is much more successful than either of the linear models.

In the end, this thesis defines a fully controlled baseline and clearly designates the limits that need to be pushed forth in the next. Our findings point to a very promising and flexible avenue for surface-code decoding – machine learning— but its engineering merits need to be demonstrated

with relentless, side-by-side comparison to optimized classical matchers. The model is an open-ended simulation framework carefully designed in this work, which provides an ideal platform to do the important follow on investigation. The conclusion and future scope is discussed in the following chapter.

Chapter 7 Conclusion & Future Scope

7.1 Conclusion

Quantum error correction (QEC) is the ultimate stepping stone between experimental quantum devices and a scalable quantum computer with fault tolerance. Operational imperfections and measurement state drift, as well as environmental noise, make raw physical qubits extremely susceptible to errors, and fault tolerance requires architectures that can isolate errors in real time. Surface codes offer an extremely promising approach which relies on applying repeated stabilizer measurements locally to encode a mapping that identifies the physical faults while preserving the fragile logical information embedded in the system. But the ultimate error suppression of any surface code is tightly linked to the computational decoder used to decode the raw syndrome patterns.

The thesis explored possibilities for data-driven error correction and compared the four decoding frameworks (Minimum-Weight Perfect Matching (MWPM), standard Logistic Regression, class-balanced Logistic Regression, and a feed-forward Multi-Layer Perceptron (MLP) in a structured computational comparison. Each decoder has been tested with a single simulation profile, a distance-3 (d=3) rotated surface-code memory circuit, repeated over three rounds of measurements and a physical error probability of 0.5%. We created a stratified partition of our core dataset (of 50,000 independent shots) into training, validation and testing sets, and our final performance metrics were obtained from a completely un-biased held-out test set of 7,500 samples.

The results from the empirical comparison of the outcomes of this face-off demonstrate the existence of an implicit hierarchy in decoding performance:

i. We had the highest performance in MWPM with the gold-standard classical matcher, with only 144 logical failures recorded, equating to a 1.92% log failure rate.
ii. The nonlinear MLP proved to be a very competitive data driven runner up with 186 failures and 2.48% logical failure rate.
iii. Standard Logistic Regression had a significant decrease in performance with 807 failures and a failure rate of 10.76%.
iv. Class-balanced Logistic Regression had the widest error profile with 1,558 failures, or a 20.77% failure rate.

These results were consistent with the fact that the MLP was able to capture complex fault networks of many detectors that our linear models were completely unable to capture. Meanwhile, the results indicate that the MLP was unable to catch up with MWPM under these basic operating conditions. The result highlights an important lesson for QEC based on data: complexity added to the neural network does not always lead to superior algorithmic performance relative to very specialized global optimization models.

This study was able to create a full computational workflow that can be reproduced. We developed an end-to-end system that generates noisy circuit streams using Stim, generates normalized feature matrices, optimizes the parameters within isolated validation sets and performs independent multi-model benchmarking. In conclusion, this thesis points out the enormous potential of nonlinear machine learning in syndrome parsing and clearly maps out the scaling limits that will need to be broken in the near future to make data-driven decoding a real possibility for future quantum architectures.

7.2 Achievement of Research Objectives.

We achieved the main goals of this research pipeline:

a) ***Surface-code simulation:*** Designed a surface-code memory layout with high fidelity and low noise in Stim and set the distance to be 3 for three cycles.
b) ***Data generation and data preparation:*** We sampled 50,000 independent simulated data points, with each data point consisting of a raw combination of the detector outputs fed into a single entry of the network, and the output of the detector being considered a logical flip as the label for a binary classification.
c) ***Execution of decoding methods:*** Optimized a classical reference using PyMatching's own implementation of MWPM, and built three separate pipelines based on data using scikit-learn.
d) ***Comparative evaluation:*** All of the decoders were put to the same, independent test partition, so that the logical failure rates were tested under the same data conditions.
e) ***Analysis of machine-learning performance:*** Empirically demonstrated that the nonlinear MLP significantly outperforms linear classification baselines, and systematically reported its performance gaps from MWPM anchor.
f) ***Determination of additional research needs:*** Defined the exact scope of our main run and specifically identified the need for multi-seed validation, large distance scaling and a full sweep of noises.

Our fundamental aims in pipeline execution and comparative benchmarking were achieved in the course of our main experiment. Determining larger engineering metrics (like multi-distance scaling thresholds and gain in real-time processing) are still the next steps in future research phases.

7.3 Research Contribution

This thesis presents a structured and unbiased graph optimization and supervised data science benchmarking ecosystem that is the main contribution. This study offers a fair evaluation ground for studying the effect of structural decoding decisions on error suppression by comparing a specialised classical matcher, two linear pipelines and a nonlinear network under the same unyielding evaluation framework.

Our results will prove useful when building the decoder's model in the future as they serve as a good baseline for the future model engineering; they show that the learning capacity of a decoder changes its error-suppression profile completely. In particular, the MLP's giant improvement over the logistic regression approaches shows that stabilizer syndrome data is rich with nonlinear relationships, which necessitates the use of deep and highly expressive blocks of modeling.

At the same time, this work makes a valuable contribution to the field by drawing a clear distinction between what is known, demonstrated, and what is still a matter of debate, a question of great quality. This study explicitly distinguishes the well-established results from the unsolved scaling laws, and provides an objective and highly systematic path forward to the future of ongoing progress in data-driven correction of quantum errors.

7.4 Future Scope

This study provides a strong foundation but there are a few critical extensions that must be made to take data-driven decoders from a proof-of-concept stage in simulation to a real quantum control stack.

7.4.1 Evaluation at Large Code Distances

We chose only a distance-3 layout for our main metric. To scale this evaluation to topological structures larger than the ones used above, we must expand the two noise configurations used to include distance-5 (d=5) and distance-7 (d=7) surface codes, while maintaining precisely the same circuit templates and noise configurations. This expansion will determine if the performance gap between MLP and MWPM is reduced, equal or increased as the code footprint increases physically.

7.4.2 Performance Across Physical Error Probabilities

Future experiments should test the decoders over a continuous systematic range of physical noise rates to fully characterize the envelope in which they operate. Measuring the resulting logical failure rates with sound confidence intervals will give a detailed and accurate understanding of the performance of data-driven classifiers in the presence of varying levels of hardware noise or when hardware approaches the limits of fault-tolerant operation.

7.4.3 Improved Machine-Learning Architectures

Future studies need to go beyond the simple feed-forward networks to investigate special architectures that will be able to replicate the physical structure of quantum devices. The 2D spatial geometry of surface codes could be utilized in the integration of Convolutional Neural Networks (CNNs) and the temporal flow of the repeated measurement rounds could be utilized in the integration of Recurrent Neural Networks (RNNs) or Transformers. These changes in design must be tested side by side in a controlled test environment before they can be assumed.

Decoder reliability is essential in a practical, real-world application. Future research should focus on a more systematic transfer learning approach: testing models learned under a given physical noise level with a completely different noise rate, or training networks with small code distances and testing with larger code geometries. These stress-tests will determine hard limits of data-driven decoding logic. To break out of a single statistical window, the dataset generation and weight initialization pipelines need to be repeated with several different random seeds. Reporting accurate statistical averages, variance distributions and paired significance tests for these parallel trials will help to reinforce the reliability and credibility of our benchmarking conclusions.

If a decoder cannot meet the real-time coherence windows of physical hardware, then high accuracy is not helpful. Future work should involve the use of an established and isolated timing protocol to evaluate processing latency and memory usage. To make machine learning pipelines a useful and real-time speedup over classic graph algorithms, the times to preprocess and the costs to infer the models must be clearly documented.

A hybrid decoder could combine machine learning with conventional decoding. For example, a learned model could estimate likely error locations or provide information to a graph-based decoder. Any proposed hybrid method would need to be implemented and evaluated against the standalone MWPM and MLP baselines.

A very promising frontier is to combine and integrate these two different paradigms into a single hybrid decoding system. A trained machine learning-based model is not a panacea for the MWPM, but it can help the traditional graph matcher to be intelligent as a pre-processing layer. For example, the neural network can be trained to predict the dynamic edge weights based on the raw syndrome patterns, detect highly complex fault clusters or direct specific local matching decisions. The result is a kind of hybrid that combines the real-time inference power of machine learning with the worldwide mathematical certainties of classical optimization, which could represent a very useful path forward to build reliable layers of quantum control.

7.5 Final Remarks

This project establishes a computational workflow for comparing MWPM, standard Logistic Regression, class-balanced Logistic Regression, and an MLP under one stated simulation configuration. In the reported test set, MWPM achieved the lowest logical failure rate, while the MLP performed better than the two linear classifiers. These results are preliminary and apply to the evaluated code distance, noise probability, and measurement-round configuration.

The results show that the MLP improved on the two linear classifiers but did not outperform MWPM in this experiment. This is a useful baseline for future work rather than evidence that machine-learning decoders cannot outperform conventional decoders in other settings.

Assessing whether machine learning can improve logical-error suppression, scaling, or real-time decoding requires further tests across code distances, physical error rates, independent random seeds, and carefully controlled runtime measurements. The workflow presented here can serve as a starting point for those investigations.

REFERENCES

[1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary ed. Cambridge, U.K.: Cambridge University Press, 2010. ISBN: 978-1-107-00217-3.

[2] P. W. Shor, "Scheme for reducing decoherence in quantum computer memory," *Physical Review A*, vol. 52, no. 4, pp. R2493–R2496, 1995.

[3] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, "Topological quantum memory," *Journal of Mathematical Physics*, vol. 43, no. 9, [pp. 4452–4505, 2002. doi: 10.1063/1.1499754].

[4] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A*, vol. 86, no. 3, [Art. no. 032324, 2012. doi: 10.1103/PhysRevA.86.032324].

[5] Google Quantum AI, "Suppressing quantum errors by scaling a surface code logical qubit," *Nature*, vol. 614, [pp. 676–681, 2023. doi: 10.1038/s41586-022-05434-1].

[6] Google Quantum AI et al., "Quantum error correction below the surface code threshold," *Nature*, vol. 638, [pp. 920–926, 2025. doi: 10.1038/s41586-024-08449-y].

[7] N. P. Breuckmann and X. Ni, "Scalable neural network decoders for higher dimensional quantum codes," *Quantum*, vol. 2, [Art. no. 68, 2018. doi: 10.22331/q-2018-05-24-68].

[8] S. Varsamopoulos, K. Bertels, and C. G. Almudever, "Decoding surface code with a distributed neural network-based decoder," *Quantum Machine Intelligence*, vol. 2, [Art. no. 3, 2020. doi: 10.1007/s42484-020-00015-9].

[9] S. Bordoni and S. Giagu, "Convolutional neural network based decoders for surface codes," *Quantum Information Processing*, vol. 22, Art. no. 151, 2023. [doi: 10.1007/s11128-023-03898-2].

[10] J. Bausch et al., "Learning high-accuracy error decoding for quantum processors," *Nature*, [vol. 635, pp. 834–840, 2024. doi: 10.1038/s41586-024-08148-8].

[11] O. Higgott, "PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching," *ACM Transactions on Quantum Computing*, vol. 3, no. 3, Art. no. 16, 2022. doi: 10.1145/3505637.

[12] C. Gidney, "Stim: A fast stabilizer circuit simulator," *Quantum*, vol. 5, Art. no. 497, 2021. doi: 10.22331/q-2021-07-06-497.

[13] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[14] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[15] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

Appendix

This appendix outlines the main implementation details adopted in comparing conventional and machine-learning-assisted surface-code decoders using a simulation. The notebook, ML_SC_Dec.ipynb, contains a longer version of the code that is included in this document for readability—this is the notebook that should be provided as a separate supplementary file

A.1 Software Environment

The simulation and analysis have been done using Python. Stim was used to generate noisy stabilizer-measurement circuits, while PyMatching gave the minimum-weight perfect matching (MWPM) baseline was implemented with scikit-learn, the logistic-regression and multilayer-perceptron (MLP) models. The arrays were used in NumPy and plotted in Matplotlib, with the exception of the simulation parameters and the circuit generation, which were performed in the other code.

```
%pip install -q stim pymatching scikit-learn numpy matplotlib

import numpy as np
import matplotlib.pyplot as plt
import stim

import pymatching
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
```

A.2 Simulation Parameters and Circuit Generation

The first one was a rotated surface-code X-memory circuit with d = 3, 3 measurement rounds, p = 0.5% (or 0.005), 50,000 simulated shots and random seed 42. The circuit noise settings were applied identically to all of the channels used in the model (depolarization, reset-flip, measurement-flip).

```
SEED = 42
DISTANCE = 3
ROUNDS = 3
TRAINING_NOISE_RATE = 0.005
DATASET_SHOTS = 50000

np.random.seed(SEED)

def make_surface_code_circuit(error_probability, distance=DISTANCE, rounds=ROUNDS):
    return stim.Circuit.generated(
        "surface_code:rotated_memory_x",
        distance=distance,
        rounds=rounds,
```

```
        after_clifford_depolarization=error_probability,
        before_round_data_depolarization=error_probability,
        after_reset_flip_probability=error_probability,
        before_measure_flip_probability=error_probability,
    )

circuit = make_surface_code_circuit(TRAINING_NOISE_RATE)
```

A.3 Dataset Generation and Target Definition

For each shot sampled, a binary detector-event vector and a logical-observable is obtained. The features fed into the detector are the vector X, the single observable bit is the binary target y. Target value 1 represents a logical-observable change of the sampled circuit. Malformed data were detected by shape and by binary-value. Shape and binary-value checks were applied to detect malformed data.

```
sampler = circuit.compile_detector_sampler(seed=SEED)
syndromes, logical_outcomes = sampler.sample(
    shots=DATASET_SHOTS,
    separate_observables=True,
)

X = np.asarray(syndromes, dtype=np.uint8)
Y = np.asarray(logical_outcomes, dtype=np.uint8)

assert X.shape[0] == Y.shape[0]
assert X.shape[1] == circuit.num_detectors
assert Y.shape[1] == circuit.num_observables
assert np.isin(X, [0, 1]).all()
assert np.isin(y, [0, 1]).all()

y = Y[:, 0].astype(np.uint8)
```

A.4 Train, Validation, and Test Separation

Using stratification and fixed random states, the 50,000 shots were divided into a 70% for training, 15% for validation, and 15% for testing. The test set was used for a last comparison. The MLP decoder was built with the help of the circuit detector error model, but not with the validation set to fit the parameters of the model.

```
indices = np.arange(len(y))
train_idx, temp_idx = train_test_split(
    indices, test_size=0.30, random_state=SEED, stratify=y
)
val_idx, test_idx = train_test_split(
    temp_idx, test_size=0.50, random_state=SEED,
    stratify=y[temp_idx]
)

X_train, y_train = X[train_idx], y[train_idx]
```

```
X_val, y_val = X[val_idx], y[val_idx]
X_test, y_test = X[test_idx], y[test_idx]
Y_test = Y[test_idx]
```

A.5 Decoder Implementations

Standardised detector features were used for the machine learning baselines. The no class weighting Logistic regression was used, while class-balanced logistic regression was used with class_weight="balanced". When using the MLP, two hidden layer networks were used with respectively 64 and 32 neurons, both with ReLU activation and Adam optimiser.

```
dem = circuit.detector_error_model(decompose_errors=True)
mwpm_decoder = pymatching.Matching.from_detector_error_model(dem)
mwpm_prediction = mwpm_decoder.decode_batch(X_test)

ordinary_lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000, class_weight=None,
                       random_state=SEED)
)
balanced_lr = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=2000, class_weight="balanced",
                       random_state=SEED)
)
mlp = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(64, 32), activation="relu",solver="adam", alpha=0.001,
max_iter=250,early_stopping=False, random_state=grid_seed)
)

ordinary_lr.fit(X_train, y_train)
balanced_lr.fit(X_train, y_train)
mlp.fit(X_train, y_train)
```

A.6 Validation-Based MLP Threshold Selection

The MLP predicts a probability of the logical-flip class. The decision threshold was determined by minimising the validation mismatch rate (using validation predictions) and in case of a tie, the threshold which is closest to 0.5 was chosen. Selected threshold was subsequently used once for evaluation of probabilities.

```
thresholds = np.arange(0.10, 0.901, 0.025)
val_prob = mlp.predict_proba(X_val)[:, 1]

best_threshold = min(
    thresholds,
    key=lambda t: ( np.mean((val_prob >= t).astype(np.uint8) != y_val), abs(t - 0.5)),
)
```

```
test_prob = mlp.predict_proba(X_test)[:, 1]
mlp_prediction = (test_prob >= best_threshold).astype(np.uint8)
```

A.7 Evaluation Metric

The main end-point is the logical-decoding failure rate, which refers to the proportion of held-out shots whose logical-observable outcome is a different one than the sample of the target. These are additional classification measures, called supplementary classification measures, which include accuracy and balanced accuracy. A 95% Wilson score interval was used to demonstrate the uncertainty in the estimated failure proportion

```
failures = int(np.sum( np.arrayas(prediction).reshape(-1) != np.asarray(actual).reshape(-1)))
total_shots = len(actual)
logical_failure_rate = failures / total_shots
```

A.8 Additional Experiments and Output Files

The notebook also includes code to perform a distance/noise grid and timing measures as well as repeated-seed comparisons and physical-error-rate sweep. These analyses should only be reported if they have been made sure that each loop contains the proper independent seeds, that all configurations succeed at the end of the run and that the CSV files saved are the last corrected run. Numerous computers or software configurations are required for both decoders, and timing should be performed on the same machine and software configuration for both.

Recommended supplementary files:

(i) ML_SC_Dec-.ipynb, containing the complete code and outputs;
(ii) requirements.txt, listing the tested package versions;
(iii) README.txt, describing execution order and expected outputs;
(iv) CSV result files generated by the final verified notebook.

A.9 Reproducibility Note and Implementation Limitation

A fixed seed enhances repeatability but not necessarily generalisation and statistical robustness. In the notebook, there is a section that accepts input of distance and output a noise grid, this section needs to be corrected before the results are taken as final, changes the loop variable grid_seed within the loop, and the variable GRID_BASE_SEED is not defined. For each intended seed, remove the reassignment and use the value provided by the loop, and rewrite and recreate the grid tables and figures. Avoid reporting distance-grid or timing results from a not-verified source or report them as "unconfirmed" results.

The following excerpts are short to make a readable report. The notebook that is submitted should be the definitive version and the versions of the software and hardware used in the last run should be stated in the report for exact reproduction.